LG Electronics

LDPL Programming Guide
Part: Logic Library and Examples

Copyright - ⓒ - 2011-LG Electronics, Inc.

AE Lab.
Corporate Technology Officer

Copyright © 2011 LG Electronics Inc. All Rights Reserved.

Though every care has been taken to ensure the accuracy of this document, LG Electronics Inc. cannot accept responsibility for any errors or omissions or for any loss occurred to any person, whether legal or natural, from acting, or refraining from action, as a result of the information contained herein. Information in this document is subject to change at any time without obligation to notify any person of such changes.

LG Electronics Inc. may have patents or patent pending applications, trademarks copyrights or other intellectual property rights covering subject matter in this document. The furnishing of this document does not give the recipient or reader any license to these patents, trademarks copyrights or other intellectual property rights.

No part of this document may be communicated, distributed, reproduced or transmitted in any form or by any means, electronic or mechanical or otherwise, for any purpose, without the prior written permission of LG Electronics Inc.

The document is subject to revision without further notice.

All brand names and product names mentioned in this document are trademarks or registered trademarks of their respective owners.

Software Information

AELab
Corporate Technology Officer
LG Electronics Inc.

327-23, Gasan-Dong, Geumchun-Gu,
Seoul, Korea 137-130

Tel: (82)-(2)-xxx-xxxx
Fax: (82)-(2)-xxx-xxxx

Contact us: junseo.son@lge.com

Document Information

AELab
Corporate Technology Officer
LG Electronics Inc.

327-23, Gasan-Dong, Geumchun-Gu,
Seoul, Korea 137-130

Tel: (82)-(2)-xxxx-xxxx
Fax: (82)-(2)-xxxx-xxxx

Contact us: junseo.son@lge.com

About This Document

Revision History

Version	Date	Comment	Author	Reviewer	Approval
0.1	2011-06-01	Initial Version	AE Lab Junseo Son Research Engineer	AE Lab Kyeungjae Kim Chief Research Engineer	AE Lab Kyeungjae Kim Chief Research Engineer
1.0	2011-06-27	How to design Operation Logic of Facility Air Handling Unit Example	AE Lab Junseo Son Research Engineer	AE Lab Kyeungjae Kim Chief Research Engineer	AE Lab Kyeungjae Kim Chief Research Engineer
1.1	2011-10-10	Add Logic Libraries	AE Lab Junseo Son Research Engineer	AE Lab Kyeungjae Kim Chief Research Engineer	AE Lab Kyeungjae Kim Chief Research Engineer
1.2	2011-10-17	Add Logic Libraries	AE Lab Junseo Son Research Engineer	AE Lab Kyeungjae Kim Chief Research Engineer	AE Lab Kyeungjae Kim Chief Research Engineer
1.3	2013-09-23	Add Logic Libraries (WRITEPOINT_PRI, RELEASE_PRI, WRITE_CURRENTRATE, READ_CURRETRATE)	AE Lab Heekyung Kim Research Engineer	AE Lab Kyeungjae Kim Chief Research Engineer	AE Lab Kyeungjae Kim Chief Research Engineer
1.4	2013-10-07	CYCLE,DUTYCYCLE,ZEB,LOADRESET 추가	박철민 J	손준서 S	김경재 C
1.5	2013-10-28	열원 에너지 절감 로직 추가.	한찬영 J	손준서 S	김경재 C
1.6	2013-11-06	WRITE_TOTALCAPA, READ_TOTALCAPA, WRITE_RUNCAPA, READ_RUNCAPA	AE Lab Heekyung Kim Research Engineer	AE Lab Kyeungjae Kim Chief Research Engineer	AE Lab Kyeungjae Kim Chief Research Engineer
1.7	2013-11-09	Add Logic Libraries(AUTO_ALTCONTROL5, AUTO_ALTCONTROL10, AUTO_ALTCONTROL14, AUTO_ALTCONTROL16)	BdMS 개발팀 이효성 연구원	BdMS 개발팀 이정화 책임	BdMS 개발팀 이정화 책임
1.8	2013-11-26	ORCONTROL, CYCLE,DUTYCYCLE,ZEB,LOADRESET 삭제	BdMS 개발팀 김민석 선임	BdMS 개발팀 이정화 책임	BdMS 개발팀 이정화 책임
1.8	2014-	물리 관제점 제어시 WAIT 사용 가이드	BdMS 개발팀	BdMS 개발팀	BdMS 개발팀

Version	Date	Comment	Author	Reviewer	Approval
	01-10		박주희 주임	이정화 책임	이정화 책임
1.8	2014-03-17	ORCONTROL 복구 함수 사용상 주의사항 추가	BdMS 개발팀 이동현 주임	BdMS 개발팀 이정화 책임	BdMS 개발팀 이정화 책임
15	2014-08-04	ALARM 함수 추가	BdMS 개발팀 이동현 주임	BdMS 개발팀 전석현 선임	BdMS 개발팀 전석현 선임
16	2014-02-11	MODBUS_MS 함수 추가	BdMS 개발팀 이동현 주임	BdMS 개발팀 권효중 책임	BdMS 개발팀 권효중 책임
17	2015-01-15	Example Phython 적용	EMS 개발팀 박준호 연구원	EMS 개발팀 민인홍책임	EMS 개발팀 민인홍 책임
18	2016-03-29	LOGIC 이름 수정	EMS 개발팀 박준호 연구원	EMS 개발팀 민인홍수석	EMS 개발팀 민인홍수석
19	2016-06-08	LOOP2, LOOP_INIT_2 로직 추가	EMS 개발팀 박준호 연구원	EMS 개발팀 민인홍수석	EMS 개발팀 민인홍수석
24	2017-08-14	NUMEROUS_ALTCONTRL, NUMEROUS_STEPCONTROL 추가	EMS 개발팀 박준호 연구원	EMS 개발팀 민인홍수석	EMS 개발팀 민인홍수석
25	2017-10-23	SREAD_LIST 추가	EMS 개발팀 박준호 연구원	EMS 개발팀 장상규 책임	EMS 개발팀 장상규 책임
26	2017-10-23	MODBUS_MS 삭제	EMS 제어개발팀 박준호 연구원	EMS 제어개발팀 장상규 책임	EMS 제어개발팀 장상규 책임

Purpose

This documentation is writed for designing programming guide of DDC Logic

Scope

This documentation includes DDC Logic Program and library for automatic control.

Audience

- Project Manager
- Project Leader
- Part Leader
- Developer
- Test Engineer
- Technical Documentation Engineer

Related Documents

Document	Comment
DDC-Logic-HLD	로직 상위설계 문서
DDC-Logic-SRS	로직 요구사항
DDC-Logic-TestCase	로직 테스트 케이스
DDC-Logic-Library-HLD	로직 라이브러리 상위설계 문서

Document	Comment
DDC-Logic-Library-SRS	로직 라이브러리 요구사항
DDC-Logic-Library-TestCase	로직 라이브러리 테스트 케이스

Organization

This Documentation is composed by 8 Chapters below.

Chapter 1 Date and Time

In Chanpter 1, Functions for Date and time

Chapter 2 Operator

In Chanpter 2, Functions for Logical Operator and Mathmetical Operator

Chapter 3 Arithmetics

In Chanpter 3, Functions of Trigger

Chapter 4 Properties

In Chanpter 4, Functions for changing properties of points

Chapter 5 Communication

In Chanpter 5, Communications between facilities and DDCs

Chapter 6 Timers

In Chanpter 6, Functions of Timer

Chapter 7 Automatic Control

In Chanpter 7, Functions for Automatic Controls and Advanced Controls

Chapter 8 Example

In Chanpter 8, Examples of Various Failicity Controls

1 Date and Time 17

1.1 CRTIME 18

1.1.1 Usage 18
1.1.2 Operation & In-out type 18
1.1.3 examples 18

1.2 DAY 19

1.2.1 fUsage 19
1.2.2 Operation & In-out type 19
1.2.3 examples 19

1.3 DAYOFM 20

1.3.1 Usage 20
1.3.2 Operation & In-out type 20
1.3.3 examples 20

1.4 MONTH 21

1.4.1 Usage 21
1.4.2 Operation & In-out type 21
1.4.3 examples 21

2 Operator 22

2.1 AND 23

2.1.1 Usage 23
2.1.2 Operation & In-out type 23
2.1.3 examples 23

2.2 OR 24

2.2.1 Usage 24
2.2.2 Operation & In-out type 24
2.2.3 examples 24

2.3 XOR 25

2.3.1 Usage 25
2.3.2 Operation & In-out type 25
2.3.3 examples 25

2.4 NOT 26

2.4.1 Usage 26
2.4.2 Operation & In-out type 26
2.4.3 examples 26

2.5 RE 27

2.5.1 Usage 27
2.5.2 Caution 27
2.5.3 Operation & In-out type 27
2.5.4 examples 27

2.6 FE 28

2.6.1 Usage 28
2.6.2 Caution 28
2.6.3 Operation & In-out type 28
2.6.4 examples 28

3 Arithmetics 29

3.1 SUM 30
3.1.1 Usage 30
3.1.2 Operation & In-out type 30
3.1.3 examples 30
3.2 AVG 32
3.2.1 Usage 32
3.2.2 Operation & In-out type 32
3.2.3 examples 32
3.3 MAX 33
3.3.1 Usage 33
3.3.2 Operation & In-out type 33
3.3.3 examples 33
3.4 MIN 35
3.4.1 Usage 35
3.4.2 Operation & In-out type 35
3.4.3 examples 35
3.5 LOG 37
3.5.1 Usage 37
3.5.2 Operation & In-out type 37
3.5.3 examples 37
3.6 EXP 38
3.6.1 Usage 38
3.6.2 Operation & In-out type 38
3.6.3 examples 38
3.7 ENT 39
3.7.1 Usage 39
3.7.2 Operation & In-out type 39
3.7.3 examples 39
3.8 ENT_NFLG 40
3.8.1 Usage 40
3.8.2 Operation & In-out type 40
3.8.3 examples 40
3.9 EQUATION 41
3.9.1 Usage 41
3.9.2 Caution 41
3.9.3 Operation & In-out type 41
3.9.4 examples 41
3.10 DEWP 42
3.10.1 Usage 42
3.10.2 Operation & In-out type 42
3.10.3 examples 42
3.11 DEWP_NFLG 43
3.11.1 Usage 43
3.11.2 Operation & In-out type 43
3.11.3 examples 43
3.12 AHTH 44
3.12.1 Usage 44

3.12.2 Operation & In-out type 44
3.12.3 examples 44
3.13 AHTRH 45
3.13.1 Usage 45
3.13.2 Operation & In-out type 45
3.13.3 examples 45
3.14 AHTRH_NFLG 46
3.14.1 Usage 46
3.14.2 Operation & In-out type 46
3.14.3 examples 46
3.15 WB 47
3.15.1 Usage 47
3.15.2 Operation & In-out type 47
3.15.3 examples 47
3.16 WB_NFLG 48
3.16.1 Usage 48
3.16.2 Operation & In-out type 48
3.16.3 examples 48
3.17 RHTX 49
3.17.1 Usage 49
3.17.2 Operation & In-out type 49
3.17.3 examples 49
3.18 RHTX_NFLG 50
3.18.1 Usage 50
3.18.2 Operation & In-out type 50
3.18.3 examples 50
3.19 RHDBWB 51
3.19.1 Usage 51
3.19.2 Operation & In-out type 51
3.19.3 examples 51
4 Properties 52
4.1 RELEASE 53
4.1.1 Usage 53
4.1.2 Operation & In-out type 53
4.1.3 examples 53
4.2 WRITE_PRI 54
4.2.1 Usage 54
4.2.2 Operation & In-out type 54
4.2.3 Examples 54
4.3 RELEASE_PRI 55
4.3.1 Usage 55
4.3.2 Operation & In-out type 55
4.3.3 examples 55
4.4 READPOINT_CURRENTRATE 56
4.4.1 Usage 56
4.4.2 Operation & In-out type 56
4.4.3 examples 56

4.5 WRITEPOINT_CURRENTRATE 57
4.5.1 Usage 57
4.5.2 Operation & In-out type 57
4.5.3 examples 57
4.6 READPOINT_ TOTALCAPA 58
4.6.1 Usage 58
4.6.2 Operation & In-out type 58
4.6.3 examples 58
4.7 WRITEPOINT_TOTALCAPA 59
4.7.1 Usage 59
4.7.2 Operation & In-out type 59
4.7.3 examples 59
4.8 READPOINT_ RUNCAPA 60
4.8.1 Usage 60
4.8.2 Operation & In-out type 60
4.8.3 examples 60
4.9 WRITEPOINT_ RUNCAPA 61
4.9.1 Usage 61
4.9.2 Operation & In-out type 61
4.9.3 examples 61
4.10 ALARM 62
4.10.1 Usage 62
4.10.2 Operation & In-out type 62
4.10.3 examples 62
4.11 READACTIVETIME 63
4.11.1 Usage 63
4.11.2 Operation & In-out type 63
4.11.3 examples 63
4.12 READFEEDBACK 64
4.12.1 Usage 64
4.12.2 Operation & In-out type 64
4.12.3 examples 64
5 Communications 65
5.1 SWRITE 66
5.1.1 Usage 66
5.1.2 Operation & In-out type 66
5.1.3 examples 66
5.2 SREAD 67
5.2.1 Usage 67
5.2.2 Operation & In-out type 67
5.2.3 examples 67
5.3 SREAD_LIST 68
5.3.1 Usage 68
5.3.2 Operation & In-out type 68
5.3.3 examples 68
~~5.4 MODBUS_MS~~ 69
~~5.4.1 Usage~~ 69
~~5.4.2 Operation & In-out type~~ 69

~~5.4.3 examples~~ 69

6 Timers 70

6.1 WAIT71

6.1.1 Usage 71
6.1.2 Caution 71
6.1.3 Operation & In-out type 71
6.1.4 examples 71

6.2 SECTIMER 72

6.2.1 Usage 72
6.2.1 Caution 72
6.2.2 Operation & In-out type 72
6.2.3 examples 72

6.3 DELAYTIME 74

6.3.1 Usage 74
6.3.2 Caution 74
6.3.3 Operation & In-out type 74
6.3.4 examples 74

7 Automatic Control 75

7.1 LOOP 76

7.1.1 Usage 76
7.1.2 Operation & In-out type 76
7.1.3 examples 77

7.2 LOOP_INIT 78

7.2.1 Usage 78
7.2.2 Operation & In-out type 78
7.2.3 examples 78

7.3 LOOP2 80

7.3.1 Usage 80
7.3.2 Operation & In-out type 80
7.3.3 examples 81

7.4 LOOP2_INIT 82

7.4.1 Usage 82
7.4.2 Operation & In-out type 82
7.4.3 examples 82

7.5 NIGHT PURGE 83

7.5.1 Usage 83
7.5.2 Operation & In-out type 83
7.5.3 examples 84

7.6 OPTIMAL START 85

7.6.1 Usage 85
7.6.2 Caution 85
7.6.3 Operation & In-out type 85
7.6.4 examples 86

7.7 OPTIMAL STOP 87

7.7.1 Usage 87
7.7.2 Caution 87

7.7.3 Operation & In-out type 87
7.7.4 examples 88
7.8 STEP CONTROL CALCULATION 89
7.8.1 Usage 89
7.8.2 Operation & In-out type 89
7.8.3 examples 90
7.9 NUMEROUS_STEPCONTROL 91
7.9.1 Usage 91
7.9.2 Operation & In-out type 91
7.9.3 examples 91
7.10 SEQUENCE CONTROL 92
7.10.1 Usage 92
7.10.2 Caution 92
7.10.3 Operation & In-out type 92
7.10.4 examples 93
7.11 ALTERNATE CONTROL 94
7.11.1 Usage 94
7.11.2 Caution 94
7.11.3 Operation & In-out type 94
7.11.4 examples 95
7.12 NUMEROUS_ALTCONTROL 97
7.12.1 Usage 97
7.12.2 Caution 97
7.12.3 Operation & In-out type 97
7.12.4 examples 97
7.13 AUTO_ALTERNATE CONTROL 99
7.13.1 Usage 99
7.13.2 Caution 99
7.13.3 Operation & In-out type 99
7.13.4 examples 100
7.13.5 연동 설비 개수별 지원 함수 101
7.14 Outdoor Reset Control 102
7.14.1 Usage 102
7.14.2 Operation & In-out type 102
7.14.3 examples 102
7.15 Chilled Water Temperature Control 104
7.15.1 Usage 104
7.15.2 Caution 104
7.15.3 Operation & In-out type 104
7.15.4 examples 105
7.16 Condenser Water Temperature Control 106
7.16.1 Usage 106
7.16.2 Caution 106
7.16.3 Operation & In-out type 106
7.16.4 examples 106
8 Sample Program 107
8.1 AHU-NORMAL-TYPE 108
8.1.1 Definition 108

8.1.2 System Design 108
8.1.3 Basic Control 108
8.1.4 Examples 109
8.2 AHU-INV-NORMAL 110
8.2.1 Definition 110
8.2.2 System Design 110
8.2.3 Basic Control 110
8.2.4 Examples 111
8.3 Chilled Water Temperature Control 116
8.4 Condenser Water Temperature Control 117

Tables

N/A

Figures

N/A

1 Date and Time

1.1 CRTIME

1.1.1 Usage

DDC내부의 시간을 출력 또는 사용할 필요가 있을 경우 사용

1.1.2 Operation & In-out type

Operation Name	CRTIME()

DDC내부의 시간을 이용하여 제어 및 감시를 하는 경우 사용하며 해당 함수를 사용하여 시간을 환산 받는다.

IN Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

OUT Arguments - Complex Type

변수명	타입	비고
output	float	시간과 분이 소수점을 기준으로 분배됨 (11 시 30 분, 11.3)

1.1.3 examples

DDC내부의 시간정보를 이용하여 설비를 제어하는 경우 (12시에 휀기동, 오후 1시에 정지)

```
engine = ddc_logic()

tempVal = engine.CRTIME()
if(tempVal == 12.0):
    engine.WRITEPOINT("Fan", 1)

if(tempVal == 13.0):
     engine.WRITEPOINT("Fan", 0)
```

1.2 DAY

1.2.1 fUsage

DDC내부의 요일을 출력 또는 사용할 필요가 있을 경우 사용

1.2.2 Operation & In-out type

Operation Name	DAY()

DDC내부의 요일을 이용하여 제어 및 감시를 하는 경우 사용하며 해당 함수를 사용하여 요일을 환산 받는다.

IN Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

OUT Arguments - Complex Type

변수명	타입	비고
output	int	요일을 숫자로 반환함 (일~토요일, 0~6)

1.2.3 examples

DDC내부의 요일 정보를 이용하여 설비를 제어하는 경우 (매주 일요일 오전 9시에는 팬기동)

```
engine = ddc_logic()

tempVal1 = engine.DAY()
tempVal2 = engine.CRTIME()

if(tempVal1 == 0):
    if(tempVal2 == 9.0):
        engine.WRITEPOINT("Fan", 1)
```

1.3 DAYOFM

1.3.1 Usage

DDC내부의 날짜를 출력 또는 사용할 필요가 있을 경우 사용

1.3.2 Operation & In-out type

Operation Name	DAYOFM()

DDC내부의 날짜을 이용하여 제어 및 감시를 하는 경우 사용하며 해당 함수를 사용하여 날짜를 환산 받는다.

IN Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

OUT Arguments - Complex Type

변수명	타입	비고
output	int	날짜를 숫자로 반환함 (1~31)

1.3.3 examples

DDC내부의 날짜 정보를 이용하여 설비를 제어하는 경우 (매달 1일 오전 9시에는 팬기동)

```
engine = ddc_logic()

tempVal1 = engine.DAYOFM()
tempVal2 = engine.CRTIME()

if(tempVal1 == 0):
    if(tempVal2 == 9.0):
        engine.WRITEPOINT("Fan", 1)
```

1.4 MONTH

1.4.1 Usage

DDC내부의 월을 출력 또는 사용할 필요가 있을 경우 사용

1.4.2 Operation & In-out type

Operation Name	MONTH()

DDC내부의 월을 이용하여 제어 및 감시를 하는 경우 사용하며 해당 함수를 사용하여 월을 환산 받는다.

IN Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

OUT Arguments - Complex Type

변수명	타입	비고
output	int	날짜를 숫자로 반환함 (1~12)

1.4.3 examples

DDC내부의 월 정보를 이용하여 설비를 제어하는 경우 (7, 8월에는 오전 6시부터 공조기 가동)

```
engine = ddc_logic()

tempVal1 = engine.MONTH()
tempVal2 =  engine.CRTIME()

if(tempVal1 == 7 or tempVal1 == 8):
    if(tempVal2 == 6.0):
        engine.WRITEPOINT("공조기", 1)
```

2 Operator

2.1 AND

2.1.1 Usage

0혹은 1의 값 간의 AND 연산을 할 경우 사용

2.1.2 Operation & In-out type

Operation Name	AND(value1,value2)

binary값을 가지는 BI, BV 값등은 그 값을 이용해서 AND 연산을 하는 경우가 있다. 이러한 연산을 통해서 일반적인 경우의 수를 제어에 사용하는 경우로서 AND 연산은 일반적인 논리 연산자의 AND 연산 기능을 제공한다.

IN Arguments - Complex Type

변수명	타입	비고
value1	int	비교 대상이 되는 값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.
Value2	int	비교 대상이 되는 값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.

OUT Arguments - Complex Type

변수명	타입	비고
output	int	연산의 결과가 반환

2.1.3 examples

설비 A의 상태와 B의 상태가 모두 ON일 경우를 계산할 경우

- value1 : value1 (A의 상태를 받아들여 내부변수를 기입)
- value2 : value2 (B의 상태를 받아들여 내부변수를 기입)

```
engine = ddc_logic()

tempVal1 = 1
tempVal2 = 0

result = engine.AND(tempVal1, tempVal2)
```

2.2 OR

2.2.1 Usage

0혹은 1의 값 간의 OR 연산을 할 경우 사용

2.2.2 Operation & In-out type

Operation Name	OR(value1,value2)

binary값을 가지는 BI, BV 값등은 그 값을 이용해서 OR 연산을 하는 경우가 있다. 이러한 연산을 통해서 일반적인 경우의 수를 제어에 사용하는 경우로서 OR 연산은 일반적인 논리 연산자의 OR 연산 기능을 제공한다.

IN Arguments - Complex Type

변수명	타입	비고
value1	int	비교 대상이 되는 값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.
Value2	int	비교 대상이 되는 값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.

OUT Arguments - Complex Type

변수명	타입	비고
output	int	연산의 결과가 반환

2.2.3 examples

설비 A의 상태와 B의 상태를 확인하여 둘중의 하나가 ON일 경우를 계산할 경우

- value1 : value1 (A의 상태를 받아들여 내부변수를 기입)
- value2 : value2 (B의 상태를 받아들여 내부변수를 기입)

```
engine = ddc_logic()

tempVal1 = 1
tempVal2 = 0

result = engine.OR(tempVal1, tempVal2)
```

2.3 XOR

2.3.1 Usage

0혹은 1의 값 간의 XOR 연산을 할 경우 사용

2.3.2 Operation & In-out type

Operation Name	XOR(value1,value2)

binary값을 가지는 BI, BV 값등은 그 값을 이용해서 XOR 연산을 하는 경우가 있다. 이러한 연산을 통해서 일반적인 경우의 수를 제어에 사용하는 경우로서 XOR 연산은 일반적인 논리 연산자의 XOR 연산 기능을 제공한다.

IN Arguments - Complex Type

변수명	타입	비고
value1	int	비교 대상이 되는 값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.
Value2	int	비교 대상이 되는 값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.

OUT Arguments - Complex Type

변수명	타입	비고
output	int	연산의 결과가 반환

2.3.3 examples

설비 A의 상태와 B의 상태를 확인하여 둘중의 하나만 ON일 경우를 계산할 경우

- value1 : value1 (A의 상태를 받아들여 내부변수를 기입)
- value2 : value2 (B의 상태를 받아들여 내부변수를 기입)

```
engine = ddc_logic()

tempVal1 = 1
tempVal2 = 0

result = engine.XOR(tempVal1, tempVal2)
```

2.4 NOT

2.4.1 Usage

0혹은 1의 값의 NOT 연산을 할 경우 사용

2.4.2 Operation & In-out type

Operation Name	NOT(value1)

binary값을 가지는 BI, BV 값등은 그 값을 이용해서 NOT 연산을 하는 경우가 있다. 이러한 연산을 통해서 일반적인 경우의 수를 제어에 사용하는 경우로서 NOT 연산은 일반적인 논리 연산자의 NOT 연산 기능을 제공한다.

IN Arguments - Complex Type

변수명	타입	비고
value1	int	비교 대상이 되는 값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.

OUT Arguments - Complex Type

변수명	타입	비고
output	int	연산의 결과가 반환

2.4.3 examples

설비 A의 상태를 확인하여 반대 되는 값일 경우의 행위를 지정하는 경우

- value1 : value1 (A의 상태를 받아들여 내부변수를 기입)

```
engine = ddc_logic()

tempVal1 = 1

result = engine.NOT(tempVal1)
```

2.5 RE

2.5.1 Usage

값이 0을 유지하다가 1로 바뀌는 순간을 조건으로 사용 할 경우 사용

2.5.2 Caution

로직 파일 당 1개의 RE 함수만 사용해야 함.
조건문에 의해 수행되지 않는 경우 비정상 동작을 할 수 있음.

2.5.3 Operation & In-out type

Operation Name	RE(point)

binary값을 가지는 BI, BV 값등은 그 값이 어떠한 상태(0)에서 다른 상태(1)로 바뀔 경우가 있다. 이러한 순간을 감지해서 추가적인 연동 동작을 사용할 경우 바뀌는 순간에 1을 반환하여 그 상태를 알려준다. 일반적인 rising edge의 동작이다.

IN Arguments - Complex Type

변수명	타입	비고
Point	Point	감지되는 관제점으로 일반적으로 binary 계열의 관제점을 사용한다.

OUT Arguments - Complex Type

변수명	타입	비고
output	int	연산의 결과가 반환(0 → 평소 상태, 1 → 상태 변환된 순간)

2.5.4 examples

배기팬의 상태가 운전 되었을 경우 배기 댐퍼를 닫음

```
engine = ddc_logic()

if (engine.RE("배기팬상태") == 0):
    "배기댐퍼" = 1
```

2.6 FE

2.6.1 Usage

값이 1을 유지하다가 0로 바뀌는 순간을 조건으로 사용 할 경우 사용

2.6.2 Caution

로직 파일 당 1개의 FE 함수만 사용해야 함.
조건문에 의해 수행되지 않는 경우 비정상 동작을 할 수 있음.

2.6.3 Operation & In-out type

Operation Name	FE(point)

binary값을 가지는 BI, BV 값등은 그 값이 어떠한 상태(1)에서 다른 상태(0)로 바뀔 경우가 있다. 이러한 순간을 감지해서 추가적인 연동 동작을 사용할 경우 바뀌는 순간에 1을 반환하여 그 상태를 알려준다. 일반적인 falling edge의 동작이다.

IN Arguments - Complex Type

변수명	타입	비고
Point	Point	감지되는 관제점으로 일반적으로 binary 계열의 관제점을 사용한다.

OUT Arguments - Complex Type

변수명	타입	비고
output	int	연산의 결과가 반환(0 → 평소 상태, 1 → 상태 변환된 순간)

2.6.4 examples

배기팬의 상태가 정지 되었을 경우 배기 댐퍼를 닫음

```
engine = ddc_logic()

if (engine.FE("배기팬상태") == 0):
    "배기댐퍼" = 0
```

3 Arithmetics

3.1 SUM

3.1.1 Usage

입력값의 전체 합산값을 반환하는 기능

3.1.2 Operation & In-out type

Operation Name	SUM(value1, value2, value3, value4, value5)

값에 대한 합산을 수행할 경우 사용되는 함수로서 전체 5가지의 값을 받아들일 수 있다.

IN Arguments - Complex Type

변수명	타입	비고
value1	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.
Value2	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.
Value3	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.
Value4	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.
Value5	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.

OUT Arguments - Complex Type

변수명	타입	비고
output	float	연산의 결과가 반환

3.1.3 examples

설비 1~5중 다 OFF있는 경우는 펌프1~2을 모두 정지하고 모두다 ON되어 있는 경우는 펌프를 모두 다 돌리고, 둘다 아닌 경우는 1번만 돌릴 경우

- value1 : value1 (설비1의 상태를 받아들여 내부변수를 기입)
- value2 : value2 (설비2의 상태를 받아들여 내부변수를 기입)
- value3 : value3 (설비3의 상태를 받아들여 내부변수를 기입)
- value4 : value4 (설비4의 상태를 받아들여 내부변수를 기입)
- value5 : value5 (설비5의 상태를 받아들여 내부변수를 기입)

```
engine = ddc_logic()

value1 = 1
value2 = 0
value3 = 1
value4 = 0
value5 = 1
tempval = engine.SUM(value1, value2, value3, value4, value5)
if (tempval == 0):
    "펌프 1" = 0
```

```
    "펌프 2" = 0
else:
    if(tempval == 5):
        "펌프 1" = 1
        "펌프 2" = 1
    else:
        "펌프 1" = 1
        "펌프 2" = 0
```

3.2 AVG

3.2.1 Usage

입력값의 전체 평균값을 반환하는 기능

3.2.2 Operation & In-out type

Operation Name	AVG(value1, value2)

값에 대한 합산을 수행할 경우 사용되는 함수로서 전체 2가지의 값을 받아들일 수 있다.

IN Arguments - Complex Type

변수명	타입	비고
value1	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.
Value2	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.

OUT Arguments - Complex Type

변수명	타입	비고
output	float	연산의 결과가 반환

3.2.3 examples

온도센서 3개에 대하여 평균 값을 구하여 5도 이하일 경우 댐퍼를 닫고 밸브(0~10V)를 OPEN할 경우

- value1 : value1 (온도센서1의 상태를 받아들여 내부변수를 기입)
- value2 : value2 (온도센서2의 상태를 받아들여 내부변수를 기입)
- value3 : value3 (온도센서3의 상태를 받아들여 내부변수를 기입)

```
engine = ddc_logic()

value1 = 7
value2 = 2
value3 = 4

tempval = engine.AVG(value1, value2)
tempval = engein.AVG(tempval, value3)
if (tempval < 5):
    "댐퍼" = 0
    "밸브" = 100
```

3.3 MAX

3.3.1 Usage

입력값중 가장 큰값을 반환하는 기능

3.3.2 Operation & In-out type

Operation Name	MAX(value1, value2, value3, value4, value5)

값 중 가장 큰값을 반환하는 함수로서 전체 5가지의 값을 받아들일 수 있다.

IN Arguments - Complex Type

변수명	타입	비고
value1	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.
Value2	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.
Value3	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.
Value4	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.
Value5	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.

OUT Arguments - Complex Type

변수명	타입	비고
output	float	연산의 결과가 반환

3.3.3 examples

온도센서 5개에 대하여 최고 값을 구하여 30도 이상일 경우 냉방 밸브를 최대로 유지함

- value1 : value1 (온도센서1의 상태를 받아들여 내부변수를 기입)
- value2 : value2 (온도센서2의 상태를 받아들여 내부변수를 기입)
- value3 : value3 (온도센서3의 상태를 받아들여 내부변수를 기입)
- value4 : value4 (온도센서4의 상태를 받아들여 내부변수를 기입)
- value5 : value5 (온도센서5의 상태를 받아들여 내부변수를 기입)

```
engine = ddc_logic()

value1 = 18
value2 = 21
value3 = 22
value4 = 27
value5 = 31

tempval = engine.MAX(value1, value2, value3, value4, value5)
```

```
if (tempval > 30):
    "냉방밸브" = 100
```

3.4 MIN

3.4.1 Usage

입력값중 가장 작은값을 반환하는 기능

3.4.2 Operation & In-out type

Operation Name	MIN(value1, value2, value3, value4, value5)

값 중 가장 작은값을 반환하는 함수로서 전체 5가지의 값을 받아들일 수 있다.

IN Arguments - Complex Type

변수명	타입	비고
value1	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.
Value2	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.
Value3	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.
Value4	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.
Value5	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.

OUT Arguments - Complex Type

변수명	타입	비고
output	float	연산의 결과가 반환

3.4.3 examples

온도센서 5개에 대하여 최저 값을 구하여 20도 이하일 경우 난방 밸브를 최대로 유지함

- value1 : value1 (온도센서1의 상태를 받아들여 내부변수를 기입)
- value2 : value2 (온도센서2의 상태를 받아들여 내부변수를 기입)
- value3 : value3 (온도센서3의 상태를 받아들여 내부변수를 기입)
- value4 : value4 (온도센서4의 상태를 받아들여 내부변수를 기입)
- value5 : value5 (온도센서5의 상태를 받아들여 내부변수를 기입)

```
engine = ddc_logic()

value1 = 15
value2 = 22
value3 = 23
value4 = 23
value5 = 25

tempval = engine.MIN(value1, value2, value3, value4, value5)
```

```
if (tempval > 20):
    "난방밸브" = 100
```

3.5 LOG

3.5.1 Usage

입력값을 자연로그의 값으로 변환하여 반환하는 기능

3.5.2 Operation & In-out type

Operation Name	LOG(value1)

입력값을 자연로그의 값으로 변환하여 반환한다.

IN Arguments - Complex Type

변수명	타입	비고
value1	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.

OUT Arguments - Complex Type

변수명	타입	비고
output	float	연산의 결과가 반환

3.5.3 examples

소음 측정센서 값에 대한 상용 로그값 계산

```
engine = ddc_logic()

tempVal = engine.READPOINT("소음측정센서")
val = engine.LOG(tempVal)
```

3.6 EXP

3.6.1 Usage

입력값의 지수 값을 반환하는 기능

3.6.2 Operation & In-out type

Operation Name	EXP(value1)

입력값의 지수 값으로 반환한다.

IN Arguments - Complex Type

변수명	타입	비고
value1	float	값으로서 포인트의 값을 내부 변수로 받아들여 그 변수를 기입해도 된다.

OUT Arguments - Complex Type

변수명	타입	비고
output	float	연산의 결과가 반환

3.6.3 examples

지수값을 계산

```
engine = ddc_logic()

val = engine.EXP(2.37)
```

3.7 ENT

3.7.1 Usage

온도와 습도를 바탕으로 공기가 가진 엔탈피 값을 계산하는 함수

3.7.2 Operation & In-out type

Operation Name	ENT(point1, point2, flag)

IN Arguments - Complex Type

변수명	타입	비고
temperature	point	온도센서가 연결된 관제점으로 관제점 타입만 가능하다.
humidity	point	습도센서가 연결된 관제점으로 관제점 타입만 가능하다.
Flag	Int	습도센서가 연결된 analog 관제점의 타입이다. 0→0~5V, 1→0~10V, 2→4~20mA

OUT Arguments - Complex Type

변수명	타입	비고
output	float	연산의 결과가 반환

3.7.3 examples

외기 온도와 습도 센서(0~10V)를 이용하여 외기의 엔탈피를 계산

```
engine = ddc_logic()

val = engine.ENT("외기온도", "외기습도", 1)
```

3.8 ENT_NFLG

3.8.1 Usage

온도와 습도를 바탕으로 공기가 가진 엔탈피 값을 계산하는 함수

3.8.2 Operation & In-out type

Operation Name	ENT_NFLG(point1, point2)

IN Arguments - Complex Type

변수명	타입	비고
temperature	point	온도센서가 연결된 관제점으로 관제점 타입만 가능하다.
humidity	point	상대습도 값 관제점

OUT Arguments - Complex Type

변수명	타입	비고
output	float	연산의 결과가 반환

3.8.3 examples

외기 온도와 습도 센서(0~10V)를 이용하여 외기의 엔탈피를 계산

```
engine = ddc_logic()

val = engine.ENT_NFLG("외기온도", "외기습도")
```

$ENT(“외기온도”,”외기습도”);

3.9 EQUATION

3.9.1 Usage

X,Y축의 값을 토대로 1차방정식을 구성하여 계산하는 함수

3.9.2 Caution

입력 파라미터는 x0 가 x1 보다 작은 값이 되도록 해야 함

3.9.3 Operation & In-out type

Operation Name	EQUATION(point, x0,y0,x1,y1)

IN Arguments - Complex Type

변수명	타입	비고
x-cordinates	point	현재 X 축에 해당하는 값
x0	double	X 축의 값중 알려진 값
y0	double	x0 의 값에 해당하는 y 축의 값
x1	double	X 축의 값중 알려진 값
y1	double	x1 의 값에 해당하는 y 축의 값

OUT Arguments - Complex Type

변수명	타입	비고
output	double	방정식의 연산 결과

3.9.4 examples

설정온도가 -5일 경우 제어 값이 0이고 설정온도가 -1일 경우 제어값이 4가 출력이 되야 하고 해당 값들의 관계는 1차방정식의 관계를 가질 경우

```
engine = ddc_logic()

val = engine.EQUATION("설정온도", -1, 4, 5, 0)
```

3.10 DEWP

3.10.1 Usage

온도와 상대습도를 바탕으로 노점온도를 구하는 함수

3.10.2 Operation & In-out type

Operation Name	DEWP(temp, rh, flag)

입력값의 지수 값으로 반환한다.

IN Arguments - Complex Type

변수명	타입	비고
temp	point	온도센서가 연결된 관제점
rh	point	습도센서가 연결된 관제점
Flag	int	습도센서의 타입 0~5v 타입인 경우 flag = 0 0~10v 타입인 경우 flag = 1 4~20mA 타입인 경우 flag = 2

OUT Arguments - Complex Type

변수명	타입	비고
output	double	연산 결과

3.10.3 examples

외기 온도와 습도 센서(0~10V)를 이용하여 노점온도를 구함

```
engine = ddc_logic()

val = engine.DEWP("외기온도", "외기습도", 1)
```

3.11 DEWP_NFLG

3.11.1 Usage

온도와 상대습도를 바탕으로 노점온도를 구하는 함수

3.11.2 Operation & In-out type

Operation Name	DEWP_NFLG(temp, rh)

입력값의 지수 값으로 반환한다.

IN Arguments - Complex Type

변수명	타입	비고
temp	point	온도센서가 연결된 관제점
rh	point	습도값 관제점

OUT Arguments - Complex Type

변수명	타입	비고
output	double	연산 결과

3.11.3 examples

외기 온도와 습도를 이용하여 노점온도를 구함

```
engine = ddc_logic()

val = engine.DEWP_NFLG("외기온도", "외기습도")
```

3.12 AHTH

3.12.1 Usage

온도와 엔탈피를 바탕으로 절대습도를 구하는 함수

3.12.2 Operation & In-out type

Operation Name	AHTH(temp, enthalpy)

IN Arguments - Complex Type

변수명	타입	비고
temp	point	온도센서가 연결된 관제점
Enthalpy	point	엔탈피 값 관제점

OUT Arguments - Complex Type

변수명	타입	비고
output	double	연산 결과

3.12.3 examples

외기 온도와 습도 엔탈피값을 이용하여 절대습도 구하기

```
engine = ddc_logic()

val = engine.AHTH("외기온도", "엔탈피")
```

3.13 AHTRH

3.13.1 Usage

온도와 상대습도를 이용하여 절대습도를 구하는 함수

3.13.2 Operation & In-out type

Operation Name	AHTRH (temp, rh, flag)

IN Arguments - Complex Type

변수명	타입	비고
temp	point	온도센서가 연결된 관제점
rh	point	습도센서가 연결된 관제점
flag	int	습도센서의 타입 0~5v 타입인 경우 flag = 0 0~10v 타입인 경우 flag = 1 4~20mA 타입인 경우 flag = 2

OUT Arguments - Complex Type

변수명	타입	비고
output	double	연산 결과

3.13.3 examples

습도센서 0~10v 타입인 경우

```
engine = ddc_logic()

val = engine.AHTRH("외기온도", "습도", 1)
```

3.14 AHTRH_NFLG

3.14.1 Usage

온도와 상대습도를 이용하여 절대습도를 구하는 함수

3.14.2 Operation & In-out type

Operation Name	AHTRH_NFLG (temp, rh)

IN Arguments - Complex Type

변수명	타입	비고
temp	point	온도센서가 연결된 관제점
rh	point	습도센서 값 관제점

OUT Arguments - Complex Type

변수명	타입	비고
output	double	연산 결과

3.14.3 examples

```
engine = ddc_logic()

val = engine.AHTRH_NFLG("외기온도", "습도")
```

3.15 WB

3.15.1 Usage

온도와 상대습도를 이용하여 습구온도를 구하는 함수

3.15.2 Operation & In-out type

Operation Name	WB(temp, rh, flag)

IN Arguments - Complex Type

변수명	타입	비고
temp	point	온도센서가 연결된 관제점
rh	point	습도센서 연결된 관제점
flag	int	습도센서의 타입 0~5v 타입인 경우 flag = 0 0~10v 타입인 경우 flag = 1 4~20mA 타입인 경우 flag = 2

OUT Arguments - Complex Type

변수명	타입	비고
output	double	연산 결과

3.15.3 examples

습도센서가 0~10v 타입인경우

```
engine = ddc_logic()

val = engine.WP("외기온도", "습도", 1)
```

3.16 WB_NFLG

3.16.1 Usage

온도와 상대습도를 이용하여 습구온도를 구하는 함수

3.16.2 Operation & In-out type

Operation Name	WB_NFLG(temp, rh)

IN Arguments - Complex Type

변수명	타입	비고
temp	point	온도센서가 연결된 관제점
rh	point	습도센서 연결된 관제점

OUT Arguments - Complex Type

변수명	타입	비고
output	double	연산 결과

3.16.3 examples

습도센서가 0~10v 타입인경우

```
engine = ddc_logic()

val = engine.WP_NFLG("외기온도", "습도")
```

3.17 RHTX

3.17.1 Usage

온도와 절대습도를 이용하여 상대습도를 구하는 함수

3.17.2 Operation & In-out type

Operation Name	RHTX (temp, ah,flag)

IN Arguments - Complex Type

변수명	타입	비고
temp	point	온도센서가 연결된 관제점
rh	point	습도센서 연결된 관제점
flag	int	습도센서의 타입 0~5v 타입인 경우 flag = 0 0~10v 타입인 경우 flag = 1 4~20mA 타입인 경우 flag = 2

OUT Arguments - Complex Type

변수명	타입	비고
output	double	연산 결과

3.17.3 examples

습도센서가 0~10v 타입인경우

```
engine = ddc_logic()

val = engine.RHTX("외기온도", "습도", 1)
```

3.18 RHTX_NFLG

3.18.1 Usage

온도와 절대습도를 이용하여 상대습도를 구하는 함수

3.18.2 Operation & In-out type

Operation Name	RHTX_NFLG (temp, ah)

IN Arguments - Complex Type

변수명	타입	비고
temp	point	온도센서가 연결된 관제점
rh	point	절대습도 값 관제점

OUT Arguments - Complex Type

변수명	타입	비고
output	double	연산 결과

3.18.3 examples

습도센서가 0~10v 타입인경우

```
engine = ddc_logic()

val = engine.RHTX_NFLG("외기온도", "습도")
```

3.19 RHDBWB

3.19.1 Usage

건구온도와 습구온도를 이용하여 상대습도를 구하는 함수

3.19.2 Operation & In-out type

Operation Name	RHDBWB (dryTemp, wetTemp)

IN Arguments - Complex Type

변수명	타입	비고
dryTemp	point	건구온도 관제점
wetTemp	point	습구온도 관제점

OUT Arguments - Complex Type

변수명	타입	비고
output	double	연산 결과

3.19.3 examples

```
engine = ddc_logic()

val = engine.RHDBWP("건구온도", "습구온도")
```

4 Properties

4.1 RELEASE

4.1.1 Usage

관제점이 가지는 속성중의 하나인 우선순위를 해제하는 함수.

4.1.2 Operation & In-out type

Operation Name	RELEASE(point)

관제점에 적용된 모든 우선순위를 헤제한다.

IN Arguments - Complex Type

변수명	타입	비고
Point	point	우선순위를 해제시킬 관제점의 이름

OUT Arguments - Complex Type

변수명	타입	비고
output	int	0 : 우선순위 해제 성공 Other : 우선순위 해제 실패 (error)

4.1.3 examples

로직으로 제어중인 급기팬 관제점을 사용자가 제어하고 싶어서 우선순위를 해제하는 경우
- point : “급기팬제어”

```
engine = ddc_logic()

engine.RELEASE("급기팬제어")
```

4.2 WRITE_PRI

4.2.1 Usage

관제점의 값을 특정 우선순위로 제어할 경우.
로직에서 관제점 값을 제어하는 경우 기본 우선순위가 5이지만, 5가 아닌 다른 우선순위로 제어하고자 할 때 사용.

4.2.2 Operation & In-out type

Operation Name	WRITE_PRI(point,value,priority)

IN Arguments - Complex Type

변수명	타입	비고
Point	point	제어할 관제점의 이름
value	float	제어 값
priority	Int	우선순위. 1~16까지 입력 가능

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

4.2.3 Examples

우선순위 7로 조명을 끄고자 할 경우

- Point : “조명1 제어”
- value : 0
- priority : 7

```
engine = ddc_logic()


engine.WRITE_PRI("조명 1 제어", 0, 7)
```

4.3 RELEASE_PRI

4.3.1 Usage

관제점의 특정 우선순위를 해제할 경우.

4.3.2 Operation & In-out type

Operation Name	RELEASE_PRI(point,priority)

IN Arguments - Complex Type

변수명	타입	비고
Point	point	관제점의 이름
priority	Int	우선순위. 1~16까지 입력 가능

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

4.3.3 examples

조명 제어 관제점의 우선순위 7을 해제하고자 할 경우
- Point : “조명1 제어”
- priority : 7

```
engine = ddc_logic()

engine.RELEASE_PRI("조명 1 제어", 7)
```

4.4 READPOINT_CURRENTRATE

4.4.1 Usage

PEAK타입 관제점의 current_rate(현재운전률) 속성 값을 읽을 경우.

4.4.2 Operation & In-out type

Operation Name	READPOINT_CURRENTRATE(point)

IN Arguments - Complex Type

변수명	타입	비고
Point	point	PEAK 타입 관제점의 이름

OUT Arguments - Complex Type

변수명	타입	비고
CurrentRate	double	현재운전률

4.4.3 examples

AC의 현재 운전률을 읽을 경우
- Point : “PEAK_AC”

```
engine = ddc_logic()


val = engine.READPOINT_CURRENTRATE("PEAK_AC")
```

4.5 WRITEPOINT_CURRENTRATE

4.5.1 Usage

PEAK타입 관제점의 current_rate(현재운전률) 속성 값을 쓸 경우.

4.5.2 Operation & In-out type

Operation Name	WRITEPOINT_CURRENTRATE(point,CurrentRate)

IN Arguments - Complex Type

변수명	타입	비고
Point	point	PEAK 타입 관제점의 이름
CurrentRate	double	현재운전률

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

4.5.3 examples

AC의 현재 운전률 50%를 관제점의 current_rate 속성에 쓸 경우
- Point : “PEAK_AC”
- CurrentRate : 50

```
engine = ddc_logic()

engine.WRITEPOINT_CURRENTRATE("PEAK_AC", 50)
```

4.6 READPOINT_ TOTALCAPA

4.6.1 Usage

각 설비별 PEAK타입 관제점의 total_capacity(전체 용량) 속성 값을 읽을 경우.

4.6.2 Operation & In-out type

Operation Name	READPOINT_ TOTALCAPA (point)

IN Arguments - Complex Type

변수명	타입	비고
Point	point	PEAK 타입 관제점의 이름

OUT Arguments - Complex Type

변수명	타입	비고
TotalCapa	double	전체 용량 (단위: W)

4.6.3 examples

AC의 전체 용량을 읽을 경우
- Point : “PEAK_AC”

```
engine = ddc_logic()

val = engine.READPOINT_TOTALCAPA("PEAK_AC")
```

4.7 WRITEPOINT_TOTALCAPA

4.7.1 Usage

각 설비별 PEAK타입 관제점의 total_capacity(전체 용량) 속성 값을 쓸 경우.

4.7.2 Operation & In-out type

Operation Name	WRITEPOINT_TOTALCAPA(point,TotalCapa)

IN Arguments - Complex Type

변수명	타입	비고
Point	point	PEAK 타입 관제점의 이름
TotalCapa	double	전체 용량 (단위: W)

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

4.7.3 examples

기타설비1의 전체 용량을 관제점의 total_capacity 속성에 쓸 경우
- Point : “PEAK_ETC1”
- TotalCapa : 5000

```
engine = ddc_logic()


engine.WRITEPOINT_TOTALCAPA("PEAK_ETC1", 5000)
```

4.8 READPOINT_ RUNCAPA

4.8.1 Usage

각 설비별 PEAK타입 관제점의 run_capacity(현재운전량) 속성 값을 읽을 경우.

4.8.2 Operation & In-out type

Operation Name	READPOINT_ RUNCAPA (point)

IN Arguments - Complex Type

변수명	타입	비고
Point	point	PEAK 타입 관제점의 이름

OUT Arguments - Complex Type

변수명	타입	비고
RunCapa	double	현재 운전 용량 (단위: W)

4.8.3 examples

AC의 현재 운전 용량을 읽을 경우
- Point : “PEAK_AC”

```
engine = ddc_logic()


val = engine.READPOINT_RUNCAPA ("PEAK_AC")
```

4.9 WRITEPOINT_ RUNCAPA

4.9.1 Usage

각 설비별 PEAK타입 관제점의 run_capacity(현재운전량) 속성 값을 쓸 경우.

4.9.2 Operation & In-out type

Operation Name	WRITEPOINT_RUNCAPA(point,RunCapa)

IN Arguments - Complex Type

변수명	타입	비고
Point	point	PEAK 타입 관제점의 이름
RunCapa	double	현재 운전 용량 (단위: W)

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

4.9.3 examples

기타설비1의 현재 운전 용량을 관제점의 run_capacity 속성에 쓸 경우
- Point : “PEAK_ETC1”
- RunCapa : 1500

```
engine = ddc_logic()


engine.WRITEPOINT_RUNCAPA("PEAK_ETC1", 1500)
```

4.10 ALARM

4.10.1 Usage

각 관제점의 알람 속성 값을 읽을 경우. 함수의 리턴을 받아서 0보다 크면 알람 상태이고 0 이면 알람 상태가 아닌 것으로 판단함.

4.10.2 Operation & In-out type

Operation Name	ALARM(point)

IN Arguments - Complex Type

변수명	타입	비고
Point	point	관제점의 이름

OUT Arguments - Complex Type

변수명	타입	비고
Alarm_state	double	0 : 알람이 없는 상태 1 : 상하한 경보 상태 2 : 목표값 경보 (DO,DI 등에서 목표값과 현재값이 일치할 때 발생) 4 : Feedback 경보 8 : 통신 경보 (COMM ALARM)

4.10.3 examples

AHU 급기 온도의 알람 속성을 확인 하는 경우
$ALARM 의 리턴 값이 0보다 크면 알람 상태임

- Point : "AHU1 급기온도"

```
engine = ddc_logic()

alarm = engine.ALARM ("AHU1 급기온도")
if(alarm > 0):
    engine.WRITEPOINT("ALARM_STATE", 1)
else:
    engine.WRITEPOINT("ALARM_STATE", 0)
```

4.11 READACTIVETIME

4.11.1 Usage

관제점 속성중 ACTIVE TIME 속성 값을 읽는 함수

4.11.2 Operation & In-out type

Operation Name	READACTIVETIME(point)

IN Arguments - Complex Type

변수명	타입	비고
Point	point	관제점의 이름

OUT Arguments - Complex Type

변수명	타입	비고
Alarm_state	double	해당 포인트의 Active time 속성 값

4.11.3 examples

```
engine = ddc_logic()

var = engine.READACTIVETIME("DV_00001")
```

4.12 READFEEDBACK

4.12.1 Usage

관제점 속성중 FEEDBACK 속성 값을 읽는 함수

4.12.2 Operation & In-out type

Operation Name	READFEEDBACK(point)

IN Arguments - Complex Type

변수명	타입	비고
Point	point	관제점의 이름

OUT Arguments - Complex Type

변수명	타입	비고
Alarm_state	double	해당 포인트의 FEEDBACK 속성 값

4.12.3 examples

```
engine = ddc_logic()


engine.WRITEPOINT("AV_00001", engine.READFEEDBACK("CPU00_DO1"))
```

5 Communications

5.1 SWRITE

5.1.1 Usage

Ehternet을 이용하여 다른 DDC에 속해있는 관제점을 제어할 경우에 사용

5.1.2 Operation & In-out type

Operation Name	SWRITE(ddc_ip,point,value~~,priority~~)

DDC간 연동동작이 필요하여 A-DDC에서 B-DDC의 관제점을 제어할 경우 사용된다. Ethernet 망을 통해 각 DDC의 IP주소로 해당 관제점을 제어 하는 명령이 호출된다.
~~현장에 설치된 DDC간의 상호 연동동작의 필요에 의해 A-DDC에서 B-DDC의 관제점을 제어할 경우 사용된다. DDC간의 ethernet 망을 통해 타 DDC의 관제점을 제어한다. 각 DDC가 가지고 있는 고유의 index를 통해서 해당 DDC에게 제어하는 명령이 호출되며 이경우 원하는 priority를 지정할 수 있다.~~

IN Arguments - Complex Type

변수명	타입	비고
ddc_ip:port	–	제어하고자 하는 관제점이 속한 DDC 의 IP 주소, port 가 사용된다.
ct_point	POINT	제어할 관제점 포인트의 이름이다.
ct_value	float	제어할 관제점에 적용할 값이다.
~~priority~~	~~int~~	~~제어시 적용시킬 priority 로서 1~9 까지 사용할 수 있고 숫자가 낮을수록 높은 우선권을 가진다.~~

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

5.1.3 examples

백엽상이 연결된 DDC에서 외기 온도 값을 다른 DDC(0)의 외기온도 관제점에 쓸 경우

- ddc_ip: “10.175.93.22:7000” (제어할 관제점이 속한 DDC의 IP 주소, 포트번호 (7000))
- ct_point : “외기온도”
- ct_value : outTemp (현재 외기온도를 읽어 들여 내부 변수로 받은 후 변수로 사용)
- ~~ct_priority : 9 (일반적으로 9 사용)~~

```
engine = ddc_logic()

outTemp = engine.READPOINT("외기온도")
engine.SWRITE("10.175.93.22:7000", "외기온도", outTemp)
```

5.2 SREAD

5.2.1 Usage

Ehternet을 이용하여 다른 DDC에 속해있는 관제점을 모니터링할 경우에 사용

5.2.2 Operation & In-out type

Operation Name	SREAD(ddc_ip,target_point,own_point)

DDC간 연동동작이 필요하여 A-DDC에서 B-DDC의 관제점 값을 모니터링 할 경우 사용된다. Ethernet 망을 통해 각 DDC의 IP주소로 해당 관제점 값을 모니터링 하는 명령이 호출된다. ~~현장에 설치된 DDC간의 상호 연동동작의 필요에 의해 A-DDC에서 B-DDC에 속해 있는 관제점의 값을 모니터링할 경우 사용된다. DDC간의 ethernet 망을 통해 타 DDC 관제점 값을 모니터링 한다. 각 DDC의 IP 주소를 통해서 해당 DDC에게 모니터링 하는 명령이 호출된다.~~

IN Arguments - Complex Type

변수명	타입	비고
ddc_ip:port	-	제어하고자 하는 관제점이 속한 DDC 의 IP 주소, port 가 사용된다.
target_point	POINT	관제점값을 읽을 대상 DDC 의 포인트 이름이다.
own_point	POINT	읽은 관제점의 값을 반영할 DDC 의 포인트 이름이다.

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

5.2.3 examples

백엽상이 연결된 DDC(5F_공조실)에서 외기 온도 값을 읽어 올경우

- ddc_IP : “10.175.93.22:7000” (제어할 관제점이 속한 DDC의 IP 주소, 포트번호 (7000))
- target_point : “외기온도”
- own_point : “AV_00000”

```
engine = ddc_logic()


engine.SREAD("10.175.93.22:7000", "외기온도", "AV_0000")
```

5.3 SREAD_LIST

5.3.1 Usage

Ehternet을 이용하여 다른 DDC에 속해있는 다수의 관제점을 모니터링할 경우에 사용

5.3.2 Operation & In-out type

Operation Name	SREAD_LIST(ddc_ip,target_point,own_point)

DDC간 연동동작이 필요하여 A-DDC에서 B-DDC의 관제점들을 모니터링할 경우 사용된다. Ethernet 망을 통해 각 DDC의 IP주소로 해당 관제점들을 모니터링 하는 명령이 호출된다. ~~현장에 설치된 DDC간의 상호 연동동작의 필요에 의해 A-DDC에서 B-DDC에 속해 있는 관제점의 값을 모니터링할 경우 사용된다. DDC간의 ethernet 망을 통해 타 DDC 관제점 값을 모니터링 한다. 각 DDC의 IP 주소를 통해서 해당 DDC에게 모니터링 하는 명령이 호출된다.~~

IN Arguments - Complex Type

변수명	타입	비고
targetIp	-	제어하고자 하는 관제점이 속한 DDC 의 IP 주소, port 가 사용된다.
targetPointList	POINT	관제점값을 읽을 대상 DDC 의 포인트 이름의 배열이다.
pointIdList	POINT	읽은 관제점의 값을 반영할 DDC 의 포인트 이름의 배열이다.

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

5.3.3 examples

백엽상이 연결된 DDC(5F_공조실)에서 외기 온도 값을 읽어 올경우

- ddc_IP : “10.175.93.22:7000” (제어할 관제점이 속한 DDC의 IP 주소, 포트번호 (7000))
- targetPointList : [“외기온도_1”, “외기온도_2”, “외기온도_3”, “외기온도_4”]
- pointIdList : ["VP_AV_00000","VP_AV_00001","VP_AV_00002","VP_AV_00003"]

```
engine = ddc_logic()
targetPointList : ["외기온도_1", "외기온도_2", "외기온도_3", "외기온도_4"]
pointIdList : ["VP_AV_00000","VP_AV_00001","VP_AV_00002","VP_AV_00003"]
engine.SREAD("10.175.93.22:7000", targetPointList, pointIdList)
```

5.4 MODBUS_MS

5.4.1 Usage

Modbus 통신에 사용되는 포트를 Enable 시킬지 Disable 시킬지 선택할 때 사용
같은 Modbus 라인에서 동시에 두 포트가 Enable 되지 않도록 주의해야 함

> **해당 로직라이브러리는 BACnet DDC 에서만 제공함**

5.4.2 Operation & In-out type

Operation Name	$MODBUS_MS(ms_info,port_num)

여러대의 DDC 가 동시에 같은 Modbus 라인에 연결이 되어 있고 이중 하나의 DDC 만 선택적으로 Master로 동작 시켜야 하는 경우 사용한다.
Master Enable/Disable 선택과 포트번호를 파라미터로 받아서 실시간으로 Enable/Disable 을 선택할 수 있다.

IN Arguments - Complex Type

변수명	타입	비고
ms_info	int	Master Enable 인 경우 1 Master Disable 인 경우 2 를 입력함
port_num	int	시리얼 채널 번호를 입력함. 단, BACnet DDC 의 경우 CH1 을 ms/tp 통신 전용으로 사용하므로 채널번호에 1 을 빼준다 DDC2.0 – CH1 : 1, CH2 : 2, CH3 : 3, CH4 : 4 BACnet DDC2.0 – CH2 : 1, CH3 : 2, CH4 : 3

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

5.4.3 examples

BACnet DDC의 CH2에 Modbus를 연결하고 자체진단 결과 정상이면 Enable 비정상이면 Disable 시키고 싶은 경우 ("M_자체진단" 이 0이면 정상)

```
engine = ddc_logic()

var = engine.READPOINT("M_자체진단")
if(var == 0):
    eninge.MODBUS_MS(1,1)
else:
```

```
engine.MODBUS_MS(2,1)
```

6 Timers

6.1 WAIT

6.1.1 Usage

일정 시간동안 대기 후 제어 할 경우 사용.
특히 다수의 물리 관제점을 동시에 제어하는 로직의 경우 100개당 1초의 wait time 지정.

6.1.2 Caution

해당 시간동안 로직이 대기하게 되며 같은 로직파일에 있는 다른 부분도 동작하지 않게됨.

6.1.3 Operation & In-out type

Operation Name	WAIT(time)

로직의 운영중 원하는 시간 만큼 정지했다가 다음 명령을 수행하는 함수

IN Arguments - Complex Type

변수명	타입	비고
time	int	milisecond 단위의 숫자

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	

6.1.4 examples

댐퍼 제어 후 60초 후에 팬 제어할 경우

```
engine = ddc_logic()

engine.WRITEPOINT("댐퍼", 1)
engine.WAIT(60000)
engine.WRITEPOINT("팬", 1)
```

6.2 SECTIMER

6.2.1 Usage

초단위의 counter로서 하나의 로직에서는 10개까지(0~9)의 타이머를 제공함

6.2.1 Caution

조건문에 걸리는 경우 동작하지 않으므로 reset 될 시점과 현재값을 읽을 시점에 정상적으로 동작하도록 로직을 작성해야 함

6.2.2 Operation & In-out type

Operation Name	SECTIMER(index, options)

DDC의 타이머를 reset하고 현재 흘러간 시간을 초단위로 반환 받는 메소드

IN Arguments - Complex Type

변수명	타입	비고
index	int	Timer의 인덱스로서 0~9까지의 index를 받아서 총 10개의 타이머를 사용할 수 있음
options	int	Timer를 reset할지 현재값을 읽어들일지 설정하는 옵션으로 0일 경우 reset, 1일 경우 현재 timer의 값

OUT Arguments - Complex Type

변수명	타입	비고
value	int	함수의 반환되는 값으로 옵션에 따라서 0일 경우 0을 반환하고 1일 경우는 현재 타이머의 값을 반환함

6.2.3 examples

휀의 상태가 ON으로 바뀐 후 60초 후에 댐퍼를 ON할 경우 (0번 타이머 사용)

```
engine = ddc_logic()

if (engine.READPOINT("팬의 상태") == 1):
    engine.SECTIMER(0,0)
    engine.WRITEPOINT("상태변경", 1)
else:
    engine.WRITEPOINT("상태변경", 1)

if(engine.READPOINT("상태변경") == 1):
    if (engine.SECTIMER(0,1) > 60):
```

```
        engine.WRITEPOINT("댐퍼", 1)
        engine.WRITEPOINT("상태변경", 0)
else:
    engine.WRITEPOINT("댐퍼", 0)
    engine.WRITEPOINT("상태변경", 0)
```

6.3 DELAYTIME

6.3.1 Usage

한 관제점의 상태를 토대로 기본 연동 동작을 설정한 후 원하는 시간만큼 delay 후에 제어를 수행함

6.3.2 Caution

조건문에 걸리는 경우 동작하지 않으므로 함수가 현재 시간을 확인 할 수 있도록 로직을 작성해야 함

6.3.3 Operation & In-out type

Operation Name	DELAYTIME(index, time, point1, point2, mode)

Point1과 point2의 상호 연동 관계를 선택하고 (mode→00,01,02,03) 선택한 모드에 따라서 point1의 값을 체크한 후 해당 값에 포함 될 때 일정 시간(time)을 기다린 후 point2를 제어하는 기능을 수행. 한 개의 로직에서 총 10개 (0~9)의 timer를 사용 가능함

IN Arguments - Complex Type

변수명	타입	비고
index	int	Timer의 인덱스로서 0~9까지의 index를 받아서 총 10개의 타이머를 사용할 수 있음
time	int	초단위의 시간
Point1	Point	기준이 되는 관제점으로 관제점 타입만 인자로 사용가능함
Point2	Point	Point1에 의해서 동작되는 관제점으로 관제점 타입만 인자로 사용 가능함
Mode	Int	Point1의 값에 따라 point2가 동작해야하는 모드를 설정함. 0→point1이 1일 경우 point2가 1로 값이 바뀜, 1→point1이 1일 경우 point2가 0로 값이 바뀜, 2→point1이 0일 경우 point2가 1로 값이 바뀜, 3→point1이 0일 경우 point2가 0로 값이 바뀜,

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	

6.3.4 examples

펌프1이 OFF 되었을 경우 20초 후에 펌프2가 ON되는 경우 (0번 타이머 사용)

```
engine = ddc_logic()


engine.DELAYTIME(0, 20, "펌프 1", "펌프 2", 2)
```

7 Automatic Control

7.1 LOOP

7.1.1 Usage

Proportional(비례), Integral(적분), Derivative(미분) 계수를 통해서 설정값과 현재값을 비교하여 설비를 제어하는 함수

7.1.2 Operation & In-out type

Operation Name	LOOP(type,pv_point,cv_point,set_point,p_gain,i_gain,d_gain,sampling_time,start_control_value,minimum_control_value,maximum_control_value,loop_id,dead_band)

제어 변수와 기준 입력 사이의 오차에 근거하여 계통의 출력이 기준 전압을 유지하도록 하는 피드백 제어의 일종으로, 비례(Proportional) 제어와 비례 적분(Proportional-Integral) 제어, 비례 미분(Proportional-Derivative) 제어를 조합한 것. P 제어(비례)는 기준 신호와 현재 신호 사이의 오차 신호에 적당한 비례 상수 이득을 곱해서 제어 신호를 만든다. I 제어(비례 적분)는 오차 신호를 적분하여 제어 신호를 만드는 적분 제어를 비례 제어에 병렬로 연결해 사용한다. D 제어(비례 미분)는 오차 신호를 미분하여 제어 신호를 만드는 미분 제어를 비례 제어에 병렬로 연결하여 사용한다. 자동화 시스템의 반응을 측정할 뿐 아니라 반응을 제어할 때도 사용되는 제어 방법이며, 온도, 압력, 유량, 회전 속도 등을 제어하기 위해 쓰인다. 한 개의 Logic에서 사용될 수 있는 LOOP문은 (LOOP문에 의해 제어되는 설비) 총 10개이다.

IN Arguments - Complex Type

변수명	타입	비고
type	int	1일경우 현재값을 설정값에 맞게 하향시켜야할 경우로서 냉방시의 밸브제어가 이에 해당한다. 0일경우 현재값을 설정값에 맞게 상향시켜야할 경우로서 난방시의 밸브제어가 이에 해당한다.
pv_point	POINT	현재값을 나타내는 포인트 이름으로 일반적으로 AI 포인트가 사용된다.
cv_point	POINT	제어의 결과에 반영되는 포인트 이름으로 일반적으로 AO 포인트가 사용된다.
set_point	POINT	pv_point와 비교가 되는 포인트로 설정값을 나타내는 포인트이름이다. 일반적으로 AV 포인트가 사용된다.
p_gain	Int	비례 gain값으로 현재값과 설정값의 에러값 (|현재값-설정값|)에 비례하는 값으로 사용된다. 제어값의 일부분을 형성하는 값으로서 p_gain값의 계산은 공식에 근거하여 산출된다.
i_gain	Int	현재값과 설정값에 대한 에러값의 일부를 누적하는데 사용하는 gain값으로 현재값이 설정값에 도달하는데에 대한 속도 및 완충에 대한 역할을 한다. p값의2%로 설정하면 적당하며 현장에 따라서 tunning을 하여야 한다.
d_gain	Int	적용되는 시스템의 응답속도를 지정하는 역할을 하는 gain값으로 보통 빠른 응답시스템에 적용된다. 현재값과 설정값에 대한 에러값에서 누적보상치 값을 뺀 값의 일부를 사용하는 값이다.
sampling_time	Int	LOOP을 수행하는 주기로 사용되며 0일 경우 기본 1초의 sampling을 수행한다.
start_control_value	Int	제어되는 값의 시작값을 의미하며 minimum과 maximum 사이의 값을 설정한다.

minimum_control_value	Int	제어되는 값의 최소값을 의미하며 최소값이하의 값이 계산될 경우 최소값으로 제어된다.
maximum_control_value	Int	제어되는 값의 최대값을 의미하며 최대값이상의 값이 계산될 경우 최대값으로 제어된다.
loop_id	Int	LOOP제어문이 사용되는 index로서 같은 제어 관제점일 경우 같은 ID를 할당받으며, 0~9까지만 설정할 수 있다.
dead_band	Int	PID계산을 수행하지 않을 에러값을 지정하는 값이다. 에러값이 dead_band값 보다 낮을 경우는 계산을 수행하지 않는다

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

7.1.3 examples

공조기제어 시 환기 온도와 설정온도를 비교하여서 냉방 밸브를 제어하고자 할 때

- LOOP제어타입 : 0 (원하는 온도를 낮추는 용도로 사용하는 경우 (냉방 제어시) 1)
- 현재온도 : “환기온도”
- 제어설비 : “냉방밸브”
- 설정온도 : “설정온도”
- P gain : 5000 (10~30도 까지 제어하는 경우 0~100%의 밸브제어가 된다고 가정할 때의 기울기 *1000)
- I gain : 100 (일반적으로 P gain의 2%)
- D gain : 설비의 응답속도에 따라서 가변적으로 다름
- Sampling time : 0 (0일경우 1초, LOOP제어에서의 계산이 수행되는 현재온도와 설정온도의 gap을 sampling하는 시간)
- Start control value : 0 (LOOP문을 통한 제어에서 시작 제어 값, 실제 Voltage/current 값을 기입)
- Minimum control value : 0 (LOOP문의 제어에서 계산된 값의 최소 제어 값, 실제 Voltage/current 값을 기입)
- Maximum control value : 10 (LOOP문의 제어에서 계산된 값의 최대 제어 값, 실제 Voltage/current 값을 기입)
- Loop ID : 0 (해당 로직에서 사용되는 LOOP문 ID)
- Dead band : 0 (Logic에서 값의 초과를 반대하기 위한 dead band 일반적으로 0)

```
engine = ddc_logic()


engine.LOOP(0, "환기온도", "냉방밸브", "설정온도", 5000, 100, 0, 0, 0, 0, 10, 0, 0)
```

7.2 LOOP_INIT

7.2.1 Usage

로직에서 사용되는 LOOP에서 계산되는 누적값을 초기화하는 함수

7.2.2 Operation & In-out type

Operation Name	LOOP_INIT(loop_id)

PID제어는 지능 적응 제어를 위해서 지속적으로 누적값을 저장하고 있다. 이러한 누적값은 시스템 적으로 제어가 정지되었을 경우에 초기화를 해주어야 한다. 한 개의 Logic에서 사용될 수 있는 LOOP_INIT문은 (LOOP문에 의해 제어되는 설비) 총 10개이다.

IN Arguments - Complex Type

변수명	타입	비고
Loop_id	int	LOOP_INIT제어문이 사용되는 index로서 같은 제어 관제점일 경우 같은 ID를 할당받으며, 0~9까지만 설정할 수 있다.

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

7.2.3 examples

공조기제어 시 해당 LOOP문을 초기화 할 경우

- loop id : 0 (해당 로직에서 사용되는 LOOP문 ID)

```
engine = ddc_logic()


engine.LOOPINIT(0)
```

7.3 LOOP2

7.3.1 Usage

Proportional(비례), Integral(적분), Derivative(미분) 계수를 통해서 설정값과 현재값을 비교하여 설비를 제어하는 함수

- LOOP 로직과의 차이는 pv와 sp의 차이가 음수일 경우에도 output 제어 신호를 만듬.

7.3.2 Operation & In-out type

Operation Name	LOOP2(type,pv_point,cv_point,set_point,p_gain,i_gain,d_gain,sampling_time,start_control_value,minimum_control_value,maximum_control_value,loop_id,dead_band)

제어 변수와 기준 입력 사이의 오차에 근거하여 계통의 출력이 기준 전압을 유지하도록 하는 피드백 제어의 일종으로, 비례(Proportional) 제어와 비례 적분(Proportional-Integral) 제어, 비례 미분(Proportional-Derivative) 제어를 조합한 것. P 제어(비례)는 기준 신호와 현재 신호 사이의 오차 신호에 적당한 비례 상수 이득을 곱해서 제어 신호를 만든다. I 제어(비례 적분)는 오차 신호를 적분하여 제어 신호를 만드는 적분 제어를 비례 제어에 병렬로 연결해 사용한다. D 제어(비례 미분)는 오차 신호를 미분하여 제어 신호를 만드는 미분 제어를 비례 제어에 병렬로 연결하여 사용한다. 자동화 시스템의 반응을 측정할 뿐 아니라 반응을 제어할 때도 사용되는 제어 방법이며, 온도, 압력, 유량, 회전 속도 등을 제어하기 위해 쓰인다. 한 개의 Logic에서 사용될 수 있는 LOOP문은 (LOOP문에 의해 제어되는 설비) 총 10개이다.

IN Arguments - Complex Type

변수명	타입	비고
type	int	1일경우 현재값을 설정값에 맞게 하향시켜야할 경우로서 냉방시의 밸브제어가 이에 해당한다. 0일경우 현재값을 설정값에 맞게 상향시켜야할 경우로서 난방시의 밸브제어가 이에 해당한다.
pv_point	POINT	현재값을 나타내는 포인트 이름으로 일반적으로 AI 포인트가 사용된다.
cv_point	POINT	제어의 결과에 반영되는 포인트 이름으로 일반적으로 AO 포인트가 사용된다.
set_point	POINT	pv_point와 비교가 되는 포인트로 설정값을 나타내는 포인트이름이다. 일반적으로 AV 포인트가 사용된다.
p_gain	Int	비례 gain값으로 현재값과 설정값의 에러값 (|현재값-설정값|)에 비례하는 값으로 사용된다. 제어값의 일부분을 형성하는 값으로서 p_gain값의 계산은 공식에 근거하여 산출된다.
i_gain	Int	현재값과 설정값에 대한 에러값의 일부를 누적하는데 사용하는 gain값으로 현재값이 설정값에 도달하는데에 대한 속도 및 완충에 대한 역할을 한다. p값의2%로 설정하면 적당하며 현장에 따라서 tunning을 하여야 한다.
d_gain	Int	적용되는 시스템의 응답속도를 지정하는 역할을 하는 gain값으로 보통 빠른 응답시스템에 적용된다. 현재값과 설정값에 대한 에러값에서 누적보상치 값을 뺀 값의 일부를 사용하는 값이다.
sampling_time	Int	LOOP을 수행하는 주기로 사용되며 0일 경우 기본 1초의 sampling을 수행한다.
start_control_value	Int	제어되는 값의 시작값을 의미하며 minimum과 maximum 사이의 값

		을 설정한다.
minimum_control_value	Int	제어되는 값의 최소값을 의미하며 최소값이하의 값이 계산될 경우 최소값으로 제어된다.
maximum_control_value	Int	제어되는 값의 최대값을 의미하며 최대값이상의 값이 계산될 경우 최대값으로 제어된다.
loop_id	Int	LOOP제어문이 사용되는 index로서 같은 제어 관제점일 경우 같은 ID를 할당받으며, 0~9까지만 설정할 수 있다.
dead_band	Int	PID계산을 수행하지 않을 에러값을 지정하는 값이다. 에러값이 dead_band값 보다 낮을 경우는 계산을 수행하지 않는다

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

7.3.3 examples

공조기제어 시 환기 온도와 설정온도를 비교하여서 냉방 밸브를 제어하고자 할 때

- LOOP제어타입 : 0 (원하는 온도를 낮추는 용도로 사용하는 경우 (냉방 제어시) 1)
- 현재온도 : “환기온도”
- 제어설비 : “냉방밸브”
- 설정온도 : “설정온도”
- P gain : 5000 (10~30도 까지 제어하는 경우 0~100%의 밸브제어가 된다고 가정할 때의 기울기 *1000)
- I gain : 100 (일반적으로 P gain의 2%)
- D gain : 설비의 응답속도에 따라서 가변적으로 다름
- Sampling time : 0 (0일경우 1초, LOOP제어에서의 계산이 수행되는 현재온도와 설정온도의 gap을 sampling하는 시간)
- Start control value : 0 (LOOP문을 통한 제어에서 시작 제어 값, 실제 Voltage/current 값을 기입)
- Minimum control value : 0 (LOOP문의 제어에서 계산된 값의 최소 제어 값, 실제 Voltage/current 값을 기입)
- Maximum control value : 10 (LOOP문의 제어에서 계산된 값의 최대 제어 값, 실제 Voltage/current 값을 기입)
- Loop ID : 0 (해당 로직에서 사용되는 LOOP문 ID)
- Dead band : 0 (Logic에서 값의 초과를 반대하기 위한 dead band 일반적으로 0)

```
engine = ddc_logic()


engine.LOOP2(0, "환기온도", "냉방밸브", "설정온도", 5000, 100, 0, 0, 0, 0, 10, 0, 0)
```

7.4 LOOP2_INIT

7.4.1 Usage

로직에서 사용되는 LOOP2에서 계산되는 누적값을 초기화하는 함수

7.4.2 Operation & In-out type

Operation Name	LOOP2_INIT(loop_id)

PID제어는 지능 적응 제어를 위해서 지속적으로 누적값을 저장하고 있다. 이러한 누적값은 시스템 적으로 제어가 정지되었을 경우에 초기화를 해주어야 한다. 한 개의 Logic에서 사용될 수 있는 LOOP2_INIT문은 (LOOP2문에 의해 제어되는 설비) 총 10개이다.

IN Arguments - Complex Type

변수명	타입	비고
Loop_id	int	LOOP2 로직에 사용되는 index로서 같은 제어 관제점일 경우 같은 ID를 할당받으며, 0~9까지만 설정할 수 있다.

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

7.4.3 examples

공조기제어 시 해당 LOOP2문을 초기화 할 경우

- loop id : 0 (해당 로직에서 사용되는 LOOP2문 ID)

```
engine = ddc_logic()


engine.LOOP2_INIT(0)
```

7.5 NIGHT PURGE

7.5.1 Usage

정해진 시간에 외기를 취입하여 내기를 설정온도에 맞게 조절할 수 있는 함수.

7.5.2 Operation & In-out type

Operation Name	NPURGE(start_time,end_time,period,out_temp,in_temp,dp_temp,out_settemp ,int_settemp,dp_settemp)

간절기 및 하절기 새벽 및 아침(공조기를 운전하지 않는 시간대)에 외기 공기를 취입해서 내기의 온도를 조절하는 야간 외기 취입 기능을 사용한다. 이는 에너지 효율 및 건물의 안정적 운영을 위해서 사용하는 기능으로 기계적인 냉방 이전에 건물 전체를 예냉하는 개념의 에너지 절감 제어 방식이다.

제어 시 외기온도, 외기 이슬점 온도, 실내 오도를 분석하여 제어를 시작한다. 일반적으로,

1. 외기가 10도보다 높을 경우
2. 외기가 실내온도보다 낮을 경우
3. 외기의 이슬점온도가 15도 보다 낮을 경우
4. 실내온도가 24도 보다 높을 경우

제어를 수행한다.

IN Arguments - Complex Type

변수명	타입	비고
start_time	POINT	Night purge 기동여부를 확인하는 시작 시간으로 분 단위의 값이 저장된다. (ex, 오전 3 시의 경우 180) 일반적으로 AV 포인트가 사용된다.
end_time	POINT	Night purge 기동여부 확인을 종료하는 시간으로 분 단위의 값이 저장된다. (ex, 오전 6 시의 경우 360) 일반적으로 AV 포인트가 사용된다.
period	POINT	시작시간과 종료시간 사이에서 night purge 기동 여부를 확인하는 주기로서 분 단위의 값이 저장된다. (ex, 30 분의 주기로 동작 할 경우 30), 일반적으로 AV 포인트가 사용된다.
out_temp	POINT	외기의 온도값을 가져오는 실제 포인트로서 일반적으로 AI 포인트가 사용된다.
in_temp	POINT	내기의 온도값을 가져오는 실제 포인트로서 일반적으로 AI 포인트가 사용된다.
dp_temp	POINT	외기의 이슬점 온도값을 가져오는 실제 포인트로서 일반적으로 AI 포인트가 사용된다.
out_settemp	POINT	외기의 설정 온도값이 저장되어 있는 포인트로서 일반적으로 AV 포인트가 사용된다.
in_settemp	POINT	내기의 설정 온도값이 저장되어 있는 포인트로서 일반적으로 AV 포인트가 사용된다.
dp_settemp	POINT	외기 이슬점 설정 온도값이 저장되어 있는 포인트로서 일반적으로 AV 포인트가 사용된다.

OUT Arguments - Complex Type

변수명	타입	비고
output	int	1 : night purge 동작 0 : night purge 정지

7.5.3 examples

냉방 시즌 동안 오전 3시부터 6시 사이에 30분 간격으로 외기 취입 운전할 경우

- start time : “N시작시간” (180, 시간을 분으로 환산하여 입력해야 함)
- ent time : “N종료시간” (360, 시간을 분으로 환산하여 입력해야 함)
- period : “N주기” (30, 시간을 분으로 환산하여 입력해야 함)
- out temp : “외기온도”
- in temp : “내기온도”
- dp temp : “외기 이슬점 온도”
- out settemp : “외기설정온도”
- in settemp : “내기설정온도”
- dp settemp : “이슬점 설정온도”

```
engine = ddc_logic()

nightPurge = engine.NPURGE("N 시작시간", "N 종료시간", "N 주기", "외기온도", "내기온도", "외기 이슬점 온도", "외기설정온도", "내기설정온도", "이슬점 설정온도")
```

7.6 OPTIMAL START

7.6.1 Usage

공조기의 운전 시작 시점에 대한 지능적 동작을 통해 최적의 기동 시점을 찾는 함수

7.6.2 Caution

로직 파일 당 1개의 OPTIMAL START 함수만 사용해야 함.
조건문에 의해 수행되지 않는 경우 비정상 동작을 할 수 있음.

7.6.3 Operation & In-out type

Operation Name	OPTSTART(in_temp,set_temp,margin,working_time,fact,ahu_ss,lead_time)

공조기의 최적 가동 시점을 현장의 상태에 맞게 계산하여 공조기를 자동으로 운전 시키는 기능으로서 사용자의 쾌적성 및 에너지 효율을 극대화 시키기 위해서 사용한다.
최초의 기동 시점은 사용자가 직접 입력 하고 이후의 운전 시점은 단위 면적당 에너지 제어 가능 시간을 계산해서 다음날 제어시 반영하는 방식으로 제어된다.

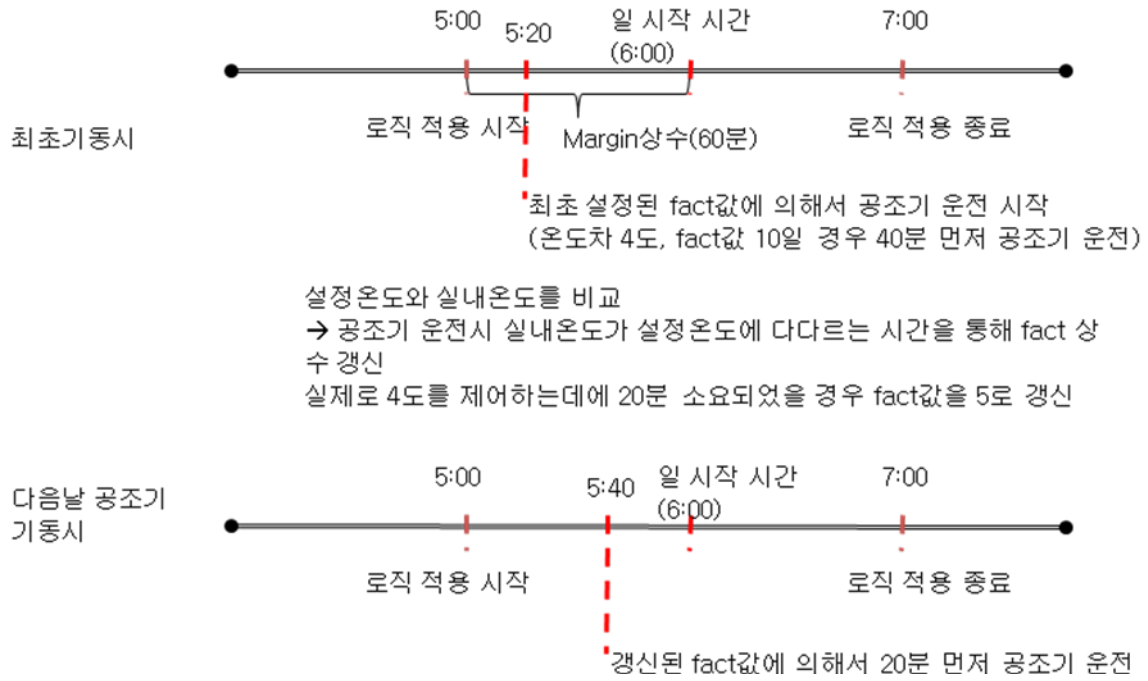


그림 7-1. 최적기동제어

IN Arguments - Complex Type

변수명	타입	비고
in_temp	POINT	공조기가 service 되는 Zone 의 실내온도에 해당하는 실제 포인트로서 일반적으로 AI 포인트가 사용된다.
set_temp	POINT	실내온도와 비교하여 공조기를 가동하는 설정온도에 해당하는 포

		인트로서 일반적으로 AV 포인트가 사용된다.
margin_time	POINT	실제 사용자가 일을 시작하는 시간에서 공조기의 가동 준비시간의 값을 지정해 optimal start 와 stop 시점을 정해주는데 사용하는 포인트로서 일반적으로 AV 포인트가 사용된다.
working_time	POINT	Service Zone 의 사용자들이 실제로 일을 시작하는 시작을 저장하는 포인트로서 일반적으로 AV 포인트가 사용된다.
factor	POINT	Service Zone 의 온도를 1 도 상승시키위한 단위 시간으로서 일반적으로 AV 포인트가 사용된다.
ahu_operation	POINT	공조기의 운전여부를 저장해 놓는 포인트로서 일반적으로 BV 포인트가 사용된다.
lead_time	POINT	계산된 공조기의 가동 적정 시점이 저장되는 포인트로서 일반적으로 AV 포인트가 사용된다.

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

7.6.4 examples

오전 6시에 일을 시작하고 최적기동제어의 초기 margin time을 1시간, factor를 1로 잡았을 경우

- in temp : “OPTSTART내기온도”
- settemp : “OPTSTART설정온도”
- margin time : “margine” (60, 시간을 분으로 환산하여 입력해야 함)
- working time : “일시작시간” (360, 시간을 분으로 환산하여 입력해야 함)
- factor : “factor” (1, 초기값은 최초에만 적용되며 추후에는 학습에 의한 계산값으로 환산됨)
- ahu_operation : “AHU운전”
- lead_time : “leadtime” (최초값은 의미가 없으며 최초운전 이후 학습에 의해서 계산, 적용됨)

```
engine = ddc_logic()

engine.OPTSTART("OPTSTART 내기온도", "OPTSTART 설정온도", "margine", "일시작시간", "factor", "AHU 운전", "leadtime")
```

7.7 OPTIMAL STOP

7.7.1 Usage

공조기의 정지 시작 시점에 대한 지능적 동작을 통해 최적의 정지 시점을 찾는 함수

7.7.2 Caution

로직 파일 당 1개의 OPTIMAL STOP 함수만 사용해야 함.
조건문에 의해 수행되지 않는 경우 비정상 동작을 할 수 있음.

7.7.3 Operation & In-out type

Operation Name	OPTSTOP(in_temp,set_temp,margin,working_time,fact,ahu_ss,lead_time)

공조기의 최적 정지 시점을 현장의 상태에 맞게 계산하여 공조기를 자동으로 운전 시키는 기능으로서 사용자의 쾌적성 및 에너지 효율을 극대화 시키기 위해서 사용한다.
최초의 정지 시점은 사용자가 직접 입력 하고 이후의 운전 시점은 단위 면적당 에너지 제어 가능 시간을 계산해서 다음날 제어시 반영하는 방식으로 제어된다.

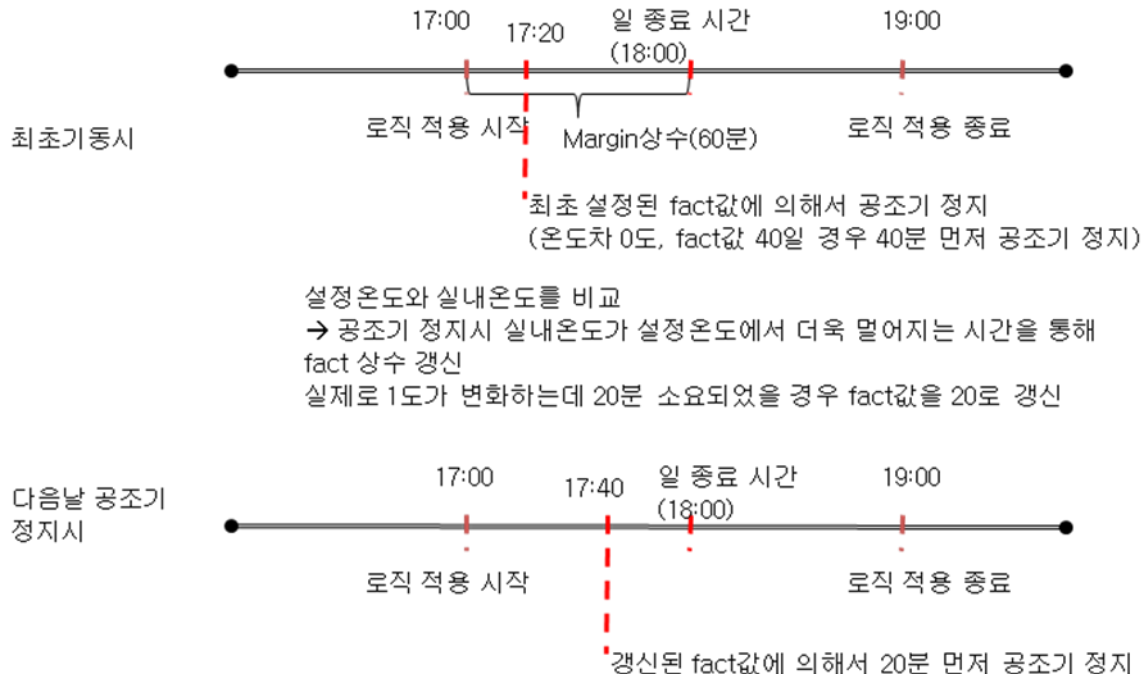


그림 7-2. 최적 정지 제어

IN Arguments - Complex Type

변수명	타입	비고
in_temp	POINT	공조기가 service 되는 Zone 의 실내온도에 해당하는 실제 포인트로서 일반적으로 AI 포인트가 사용된다.
set_temp	POINT	실내온도와 비교하여 공조기를 정지하는 설정온도에

		해당하는 포인트로서 일반적으로 AV 포인트가 사용된다.
margin_time	POINT	실제 사용자가 일을 시작하는 시간에서 공조기의 정지 준비시간의 값을 지정해 optimal start 와 stop 시점을 정해주는데 사용하는 포인트로서 일반적으로 AV 포인트가 사용된다.
working_time	POINT	Service Zone 의 사용자들이 실제로 일을 종료하는 종료를 저장하는 포인트로서 일반적으로 AV 포인트가 사용된다.
factor	POINT	Service Zone 의 온도를 1 도 상승시키위한 단위 시간으로서 일반적으로 AV 포인트가 사용된다.
ahu_operation	POINT	공조기의 운전여부를 저장해 놓는 포인트로서 일반적으로 BV 포인트가 사용된다.
lead_time	POINT	계산된 공조기의 정지 적정 시점이 저장되는 포인트로서 일반적으로 AV 포인트가 사용된다.

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

7.7.4 examples

오후 6시에 일을 끝내고 최적정지제어의 초기 margin time을 1시간, factor를 1로 잡았을 경우

- in temp : “OPTSTART내기온도”
- settemp : “OPTSTART설정온도”
- margin time : “margine” (60, 시간을 분으로 환산하여 입력해야 함)
- working time : “일종료시간” (360, 시간을 분으로 환산하여 입력해야 함)
- factor : “factor” (1, 초기값은 최초에만 적용되며 추후에는 학습에 의한 계산값으로 환산됨)
- ahu_operation : “AHU운전”
- lead_time : “leadtime” (최초값은 의미가 없으며 최초운전 이후 학습에 의해서 계산, 적용됨)

```
engine = ddc_logic()

engine.OPTSTOP("OPTSTART 내기온도", "OPTSTART 설정온도", "margine", "일시작시간", "factor", "AHU 운전", "leadtime")
```

7.8 STEP CONTROL CALCULATION

7.8.1 Usage

다수의 제어점 중 각 단계별 동작 개수를 환산하는 함수

7.8.2 Operation & In-out type

Operation Name	STEPCONTROL(supply_temp,control,hysteresis,settemp1,number1,settemp2,number2,settemp3,number3,settemp4,number4,settemp5,number5)

다수의 동일 설비를 제어 할 경우 energy saving 및 효율 적인 설비 운영을 위해서 경우에 따른 대수를 계산하여 제어한다.
운전의 기준이 되는 설정 값과 현재의 값을 비교하여 설정된 제어 대수를 계산하여 결과 값을 return해 주는 함수이다.
최대 5개의 조건에 맞는 값을 산출 할 수 있다.

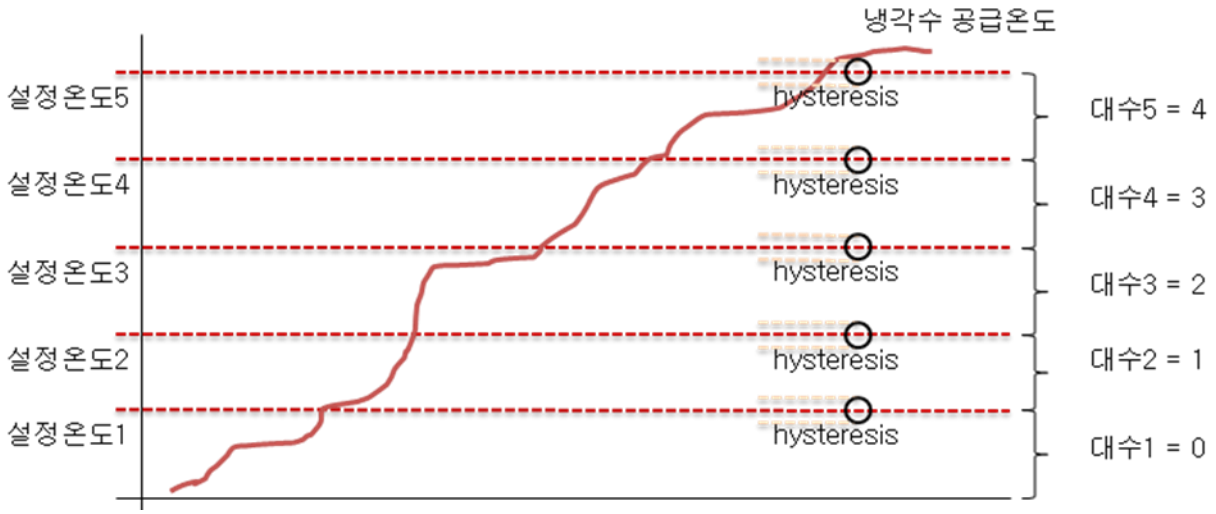


그림 7-3. 순번제어 제어 대수 계산

IN Arguments - Complex Type

변수명	타입	비고
pv_point	POINT	제어의 비교가 되는 현재값을 나타내는 포인트 이름으로 일반적으로 AI 포인트가 사용된다.
numofcontrol_point	POINT	함수의 결과 값(제어해야하는 대수)을 저장하는 포인트 이름으로 일반적으로 AV 포인트가 사용된다.
hysteresis	POINT	기준값들에 적용되는 hysteresis 로서 해당 값은 기준값들의 차이보다 클 수 없으며 사용하지 않을 경우 0 값을 가지게 된다. 일반적으로 AV 포인트가 사용된다.
set_point1	POINT	pv_point 와 비교가 되는 포인트로 설정값 1 을 나타내는 포인트이름이다. 일반적으로 AV 포인트가 사용된다.
set_num1	POINT	설정값 1 일 때의 제어 대수를 설정하는 포인트이름이다. 일반적으로 AV 포인트가 사용된다.
set_point2	POINT	pv_point 와 비교가 되는 포인트로 설정값 2 을 나타내는 포인트이름이다. 일반적으로 AV 포인트가 사용된다.

set_num2	POINT	설정값 2 일 때의 제어 대수를 설정하는 포인트이름이다. 일반적으로 AV 포인트가 사용된다.
set_point3	POINT	pv_point 와 비교가 되는 포인트로 설정값 3 을 나타내는 포인트이름이다. 일반적으로 AV 포인트가 사용된다.
set_num3	POINT	설정값 3 일 때의 제어 대수를 설정하는 포인트이름이다. 일반적으로 AV 포인트가 사용된다.
set_point4	POINT	pv_point 와 비교가 되는 포인트로 설정값 4 을 나타내는 포인트이름이다. 일반적으로 AV 포인트가 사용된다.
set_num4	POINT	설정값 4 일 때의 제어 대수를 설정하는 포인트이름이다. 일반적으로 AV 포인트가 사용된다.
set_point5	POINT	pv_point 와 비교가 되는 포인트로 설정값 5 을 나타내는 포인트이름이다. 일반적으로 AV 포인트가 사용된다.
set_num5	POINT	설정값 5 일 때의 제어 대수를 설정하는 포인트이름이다. 일반적으로 AV 포인트가 사용된다.

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

7.8.3 examples

냉동기 1대에 냉각수를 공급하는 냉각탑 3대에 대해 26~28도일 경우 1대, 28~30도일 경우 2대, 30도 이상일 경우 3대를 동작시키고자 할 경우

- supply temp : “냉각수 공급온도” (냉각탑에서 냉동기로 공급하는 온도, 온도 위치에 주의)
- control : “제어 대수”
- hysteresis : “hyst” (0.5, 온도 설정 범위의 절반 이내의 값으로 정해야함, +-hysteresis로 적용됨을 주의)
- set temp1 : “설정온도1” (26)
- number1 : “제어대수1” (0)
- set temp2 : “설정온도2” (28)
- number2 : “제어대수2” (1)
- set temp3 : “설정온도3” (30)
- number3 : “제어대수3” (2)
- set temp4 : “설정온도4” (32, 상한값을 임의로 주어 상위 개수까지 적용시킴)
- number4 : “제어대수4” (3)
- set temp5 : “설정온도4” (32, 이후의 값은 동일 값 및 개수를 적용시킴)
- number1 : “제어대수4” (3)

```
engine = ddc_logic()

engine.STEPCONTROL("냉각수 공급온도", "제어 대수", "hyst", "설정온도 1", "제어대수 1",
"설정온도 2", "제어대수 2", "설정온도 3", "제어대수 3", "설정온도 4", "제어대수 4", "설정온도 4",
"제어대수 4")
```

7.9 NUMEROUS_STEPCONTROL

7.9.1 Usage

다수의 제어점 중 각 단계별 동작 개수를 환산하는 함수

7.9.2 Operation & In-out type

Operation Name	STEPCONTROL(supply_temp,control,hysteresis, stlist,snlist)

STEP CONTROL CALCULATION 과 기능은 동일하지만 개수를 유동적으로 설정 할 수 있다.

그림 7-4. 순번제어 제어 대수 계산

IN Arguments - Complex Type

변수명	타입	비고
pv_point	POINT	제어의 비교가 되는 현재값을 나타내는 포인트 이름으로 일반적으로 AI 포인트가 사용된다.
numofcontrol_point	POINT	함수의 결과 값(제어해야하는 대수)을 저장하는 포인트 이름으로 일반적으로 AV 포인트가 사용된다.
hysteresis	POINT	기준값들에 적용되는 hysteresis 로서 해당 값은 기준값들의 차이보다 클 수 없으며 사용하지 않을 경우 0 값을 가지게 된다. 일반적으로 AV 포인트가 사용된다.
stlist	POINT	pv_point 와 비교가 되는 포인트로 설정값을 나타내는 포인트이름이다. 일반적으로 AV 포인트가 사용된다.
snlist	POINT	제어 대수를 설정하는 포인트이름이다. 일반적으로 AV 포인트가 사용된다.

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

7.9.3 examples

stlist 와 snlist 의 포인트 개수는 동일해야하며 stlist 의설정온도 1은 snlist의제어대수1로 매칭된다.

```
engine = ddc_logic()
stlist=["설정온도 1","설정온도 2","설정온도 3","설정온도 4","설정온도 5","설정온도 6",]
snlist=["제어대수 1","제어대수 2","제어대수 3","제어대수 4","제어대수 5","제어대수 6"]
engine.NUMEROUS_TEPCONTROL("냉각수 공급온도", "제어 대수", "hyst", stlist, snlist )
```

7.10 SEQUENCE CONTROL

7.10.1 Usage

다수의 제어점을 순차적으로 제어할 경우 사용하는 함수

7.10.2 Caution

로직 파일 당 1개의 SEQUENCE CONTROL 함수만 사용해야 함
(SEQCONTROL2, SEQCONTROL3을 같은 로직파일에서 사용하는 것은 가능하지만 SEQCONTROL2를 두번 사용할 수는 없음)

7.10.3 Operation & In-out type

Operation Name	SEQCONTROL(control,st1,ss1,st2,ss2,st3,ss3,st4,ss4,st5,ss5)

다수의 동일 설비를 제어 할 경우 energy saving 및 효율 적인 설비 운영을 위해서 경우에 따른 대수를 계산하여 제어한다.
계산된 대수를 가지고 전체 설비를 순차적으로 제어하는 경우에 사용되는 함수 이다.
2대부터 5대까지의 설비를 순차적으로 제어하는 함수로서 대수에 맞게 처음부터 순차적으로 ON, OFF 된다.

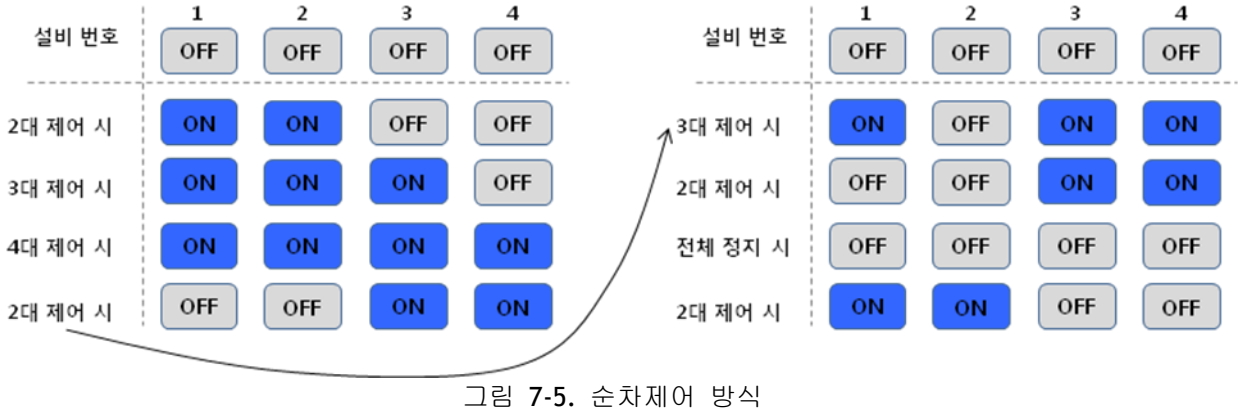


그림 7-5. 순차제어 방식

IN Arguments - Complex Type

변수명	타입	비고
numofcontrol_point	POINT	함수의 결과 값(제어해야하는 대수)을 저장하는 포인트 이름으로 일반적으로 AV 포인트가 사용된다.
st_point1	POINT	1 번 설비의 상태 포인트이름으로 일반적으로 BI 포인트가 사용된다.
sst_point1	POINT	1 번 설비의 제어 포인트이름으로 일반적으로 BO 포인트가 사용된다.
st_point2	POINT	2 번 설비의 상태 포인트이름으로 일반적으로 BI 포인트가 사용된다.
sst_point2	POINT	2 번 설비의 제어 포인트이름으로 일반적으로 BO 포인트가

		사용된다.
st_point3	POINT	3 번 설비의 상태 포인트이름으로 일반적으로 BI 포인트가 사용된다.
sst_point3	POINT	3 번 설비의 제어 포인트이름으로 일반적으로 BO 포인트가 사용된다.
st_point4	POINT	4 번 설비의 상태 포인트이름으로 일반적으로 BI 포인트가 사용된다.
sst_point4	POINT	4 번 설비의 제어 포인트이름으로 일반적으로 BO 포인트가 사용된다.
st_point5	POINT	5 번 설비의 상태 포인트이름으로 일반적으로 BI 포인트가 사용된다.
sst_point5	POINT	5 번 설비의 제어 포인트이름으로 일반적으로 BO 포인트가 사용된다.

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

7.10.4 examples

냉동기 1대에 냉각수를 공급하는 냉각탑 3대에 대해 26~28도일 경우 1대, 28~30도일 경우 2대, 30도 이상일 경우 3대를 동작시키고자 할 경우

- control : "제어 대수" (STEP CONTROL 함수에 의해 나온 결과값을 가지고 있는 제어 대수 관제점)
- st1 : "냉각탑상태1"
- ss1 : "냉각탑제어1"
- st2 : "냉각탑상태2"
- ss2 : "냉각탑제어2"
- st3 : "냉각탑상태3"
- ss3 : "냉각탑제어3"

```
engine = ddc_logic()

engine.SEQCONTROL3("제어 대수", "냉각탑상태 1", "냉각탑제어 1", "냉각탑상태 2", "냉각탑제어 2", "냉각탑상태 3", "냉각탑제어 3")
```

7.11 ALTERNATE CONTROL

7.11.1 Usage

다수의 제어점을 이전 동작을 고려하여 교번으로 제어할 경우 사용하는 함수

7.11.2 Caution

로직 파일 당 1개의 ALTERNATE CONTROL 함수만 사용해야 함
(ALTCONTROL2, ALTCONTROL 3을 같은 로직파일에서 사용하는 것은 가능하지만 ALTCONTROL 2를 두번 사용할 수는 없음)

7.11.3 Operation & In-out type

Operation Name	ALTCONTROL(control,st1,ss1,st2,ss2,st3,ss3,st4,ss4,st5,ss5)

다수의 동일 설비를 제어 할 경우 energy saving 및 효율 적인 설비 운영을 위해서 경우에 따른 대수를 계산하여 제어한다.
제어 시 특정 설비의 설비 노후 및 마모를 방지하기 위해서 이전 상태값을 바탕으로 교번제어를 하는 경우에 사용되는 함수이다.
2대부터 5대까지의 설비를 교차적 순번으로 제어하는 함수로서 대수에 맞게 이전 상태를 고려하여 해당 설비가 ON, OFF 되게 된다.

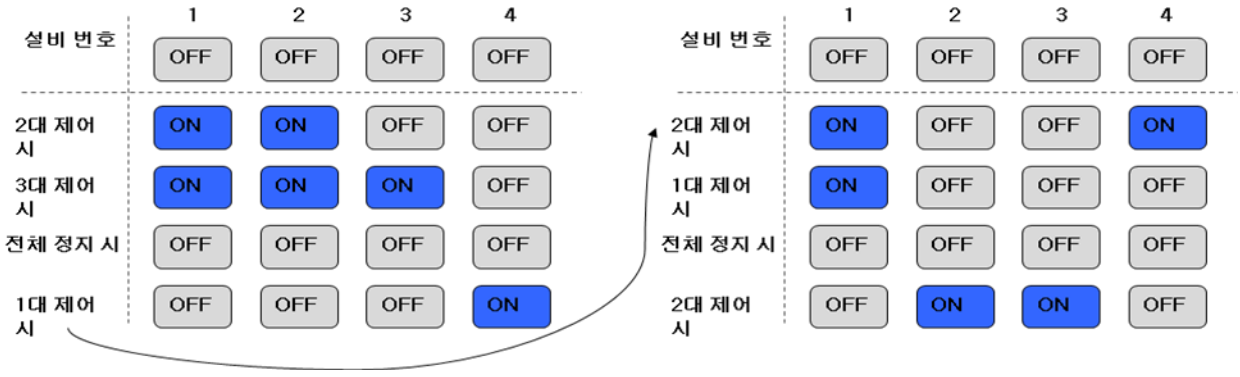


그림 7-6. 교번제어 방식

IN Arguments - Complex Type

변수명	타입	비고
numofcontrol_point	POINT	함수의 결과 값(제어해야하는 대수)을 저장하는 포인트 이름으로 일반적으로 AV 포인트가 사용된다.
st_point1	POINT	1 번 설비의 상태 포인트이름으로 일반적으로 BI 포인트가 사용된다.
sst_point1	POINT	1 번 설비의 제어 포인트이름으로 일반적으로 BO 포인트가 사용된다.
st_point2	POINT	2 번 설비의 상태 포인트이름으로 일반적으로 BI 포인트가 사용된다.

sst_point2	POINT	2 번 설비의 제어 포인트이름으로 일반적으로 BO 포인트가 사용된다.
st_point3	POINT	3 번 설비의 상태 포인트이름으로 일반적으로 BI 포인트가 사용된다.
sst_point3	POINT	3 번 설비의 제어 포인트이름으로 일반적으로 BO 포인트가 사용된다.
st_point4	POINT	4 번 설비의 상태 포인트이름으로 일반적으로 BI 포인트가 사용된다.
sst_point4	POINT	4 번 설비의 제어 포인트이름으로 일반적으로 BO 포인트가 사용된다.
st_point5	POINT	5 번 설비의 상태 포인트이름으로 일반적으로 BI 포인트가 사용된다.
sst_point5	POINT	5 번 설비의 제어 포인트이름으로 일반적으로 BO 포인트가 사용된다.

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

7.11.4 examples

냉동기 1대에 냉각수를 공급하는 냉각탑 3대에 대해 26~28도일 경우 1대, 28~30도일 경우 2대, 30도 이상일 경우 3대를 동작시키고자 할 경우

- control : "제어 대수" (STEP CONTROL 함수에 의해 나온 결과값을 가지고 있는 제어 대수 관제점)
- st1 : "냉각탑상태1"
- ss1 : "냉각탑제어1"
- st2 : "냉각탑상태2"
- ss2 : "냉각탑제어2"
- st3 : "냉각탑상태3"
- ss3 : "냉각탑제어3"

```
engine = ddc_logic()

engine.ALTCONTROL3("제어 대수", "냉각탑상태 1", "냉각탑제어 1", "냉각탑상태 2", "냉각탑제어 2", "냉각탑상태 3", "냉각탑제어 3")
```

7.12 NUMEROUS_ALTCONTROL

7.12.1 Usage

다수의 제어점을 이전 동작을 고려하여 교번으로 제어할 경우 사용하는 함수

7.12.2 Caution

로직 파일 당 1개의 NUMEROUS_ALTCONTROL 함수만 사용해야 함

7.12.3 Operation & In-out type

Operation Name	ALTCONTROL(control, splist, ssplist)

ALTCONTROL 함수와 기능적으로 동일하지만 개수의 설정을 유동적으로 설정할 수 있다.

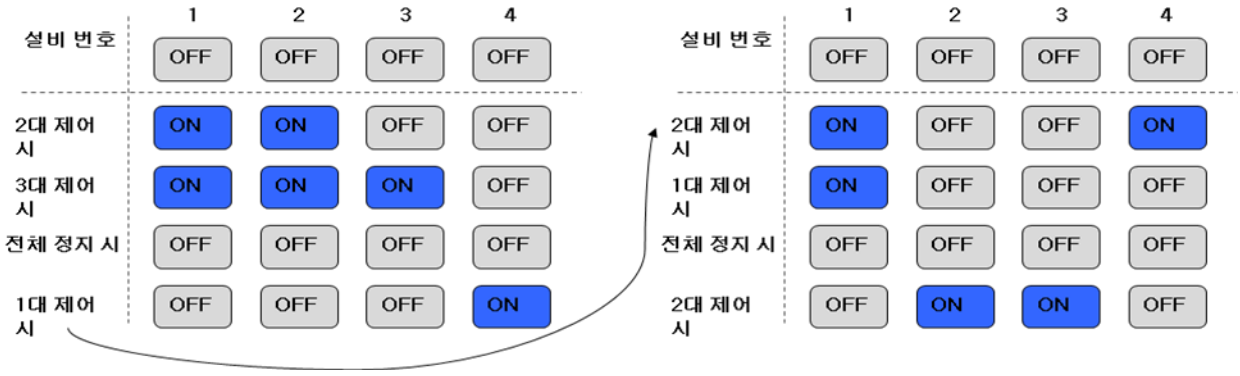


그림 7-7. 교번제어 방식

IN Arguments - Complex Type

변수명	타입	비고
numofcontrol_point	POINT	함수의 결과 값(제어해야하는 대수)을 저장하는 포인트 이름으로 일반적으로 AV 포인트가 사용된다.
splist	POINT	설비의 상태 포인트이름으로 일반적으로 BI 포인트가 사용된다.
ssplist	POINT	설비의 제어 포인트이름으로 일반적으로 BO 포인트가 사용된다.

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

7.12.4 examples

NUMEROUS_ALTCONTROL 함수의 매개변수인 splist, ssplist의 개수는 반드시 동일 해야하며 splist 의

첫번째 포인트는 **ssplist** 의 첫번째 포인트와 매칭된다.

```
engine = ddc_logic()
splist = ["냉각탑상태 1","냉각탑상태 2","냉각탑상태 3","냉각탑상태 4"]
ssplist = ["냉각탑제어 1","냉각탑제어 2", "냉각탑제어 3", "냉각탑제어 4"]
engine. NUMEROUS_ALTCONTROL("제어 대수", splist, ssplist)
```

7.13 AUTO_ALTERNATE CONTROL

7.13.1 Usage

다수의 제어점을 이전 동작을 고려하여 사용자가 설정한 시간주기에 맞게 자동으로 교번 제어할 경우 사용하는 함수

7.13.2 Caution

조건문에 의해 수행되지 않는 경우 비정상 동작을 할 수 있음.
로직 파일 당 1개의 ALTERNATE CONTROL 함수만 사용해야 함
(ALTCONTROL2, ALTCONTROL 3을 같은 로직파일에서 사용하는 것은 가능하지만 ALTCONTROL 2를 두번 사용할 수는 없음)

7.13.3 Operation & In-out type

Operation Name	AUTO_ALTCONTROL(control,st1,ss1,st2,ss2,st3,ss3,st4,ss4,st5,ss5, user1,user2,user3,user4,user5,times)

다수의 동일 설비를 사용자가 정한 시간 주기와 사용자의 설정에 따라 제어 할 경우 energy saving 및 효율 적인 설비 운영을 위해서 경우에 따른 대수를 계산하여 제어한다.
제어 시 특정 설비의 설비 노후 및 마모를 방지하기 위해서 이전 상태값을 바탕으로 사용자가 설정한 시간 및 사용여부값을 반영하여 교번제어를 하는 경우에 사용되는 함수이다.
5대,10대,14대,16대의 설비를 교차적 순번으로 제어하는 함수로서 대수에 맞게 이전 상태를 고려하여 해당 설비가 ON, OFF 되게 된다.
사용자는 자신이 사용하고자 하는 설비를 선택할 수 있으며 사용하도록 설정되어 있는 설비에 한해서 교번제어가 실시되며 사용자가 정해놓은 시간 주기별로 이전상태를 고려하여 대수에 맞게 해당 설비가 ON,OFF 되게 된다.
(기존 교번제어와 동작하는 방식은 동일하며 사용자가 각 설비를 사용할지 여부를 세팅하는 부분과 시간주기별로 대수에 맞게 다음 설비들이 자동적으로 실행되는 부분에 차이가 있음)
*시간 주기는 분단위로 설정 가능

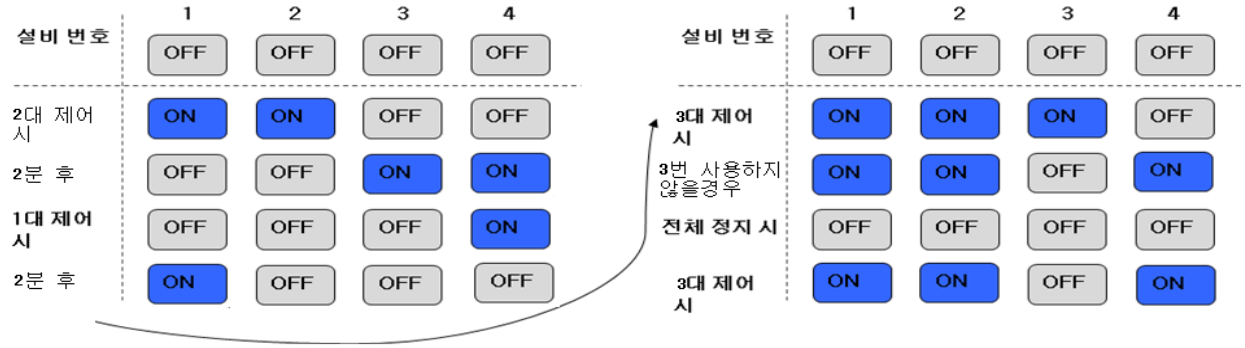


그림 7-6. 자동교번제어 방식

IN Arguments - Complex Type

변수명	타입	비고
numofcontrol_point	POINT	함수의 결과 값(제어해야하는 대수)을 저장하는 포인트 이름으로 일반적으로 AV 포인트가 사용된다.

st_point1	POINT	1 번 설비의 상태 포인트이름으로 일반적으로 BI 포인트가 사용된다.
sst_point1	POINT	1 번 설비의 제어 포인트이름으로 일반적으로 BO 포인트가 사용된다.
st_point2	POINT	2 번 설비의 상태 포인트이름으로 일반적으로 BI 포인트가 사용된다.
sst_point2	POINT	2 번 설비의 제어 포인트이름으로 일반적으로 BO 포인트가 사용된다.
st_point3	POINT	3 번 설비의 상태 포인트이름으로 일반적으로 BI 포인트가 사용된다.
sst_point3	POINT	3 번 설비의 제어 포인트이름으로 일반적으로 BO 포인트가 사용된다.
st_point4	POINT	4 번 설비의 상태 포인트이름으로 일반적으로 BI 포인트가 사용된다.
sst_point4	POINT	4 번 설비의 제어 포인트이름으로 일반적으로 BO 포인트가 사용된다.
st_point5	POINT	5 번 설비의 상태 포인트이름으로 일반적으로 BI 포인트가 사용된다.
sst_point5	POINT	5 번 설비의 제어 포인트이름으로 일반적으로 BO 포인트가 사용된다.
user1	POINT	1 번 설비의 사용유무를 세팅하는 포인트이름으로 일반적으로 BV 포인트가 사용된다.
user2	POINT	2 번 설비의 사용유무를 세팅하는 포인트이름으로 일반적으로 BV 포인트가 사용된다.
user3	POINT	3 번 설비의 사용유무를 세팅하는 포인트이름으로 일반적으로 BV 포인트가 사용된다.
user4	POINT	4 번 설비의 사용유무를 세팅하는 포인트이름으로 일반적으로 BV 포인트가 사용된다.
user5	POINT	5 번 설비의 사용유무를 세팅하는 포인트이름으로 일반적으로 BV 포인트가 사용된다.
times	POINT	사용자가 사용할 시간주기 값을 저장하는 포인트 이름으로 일반적으로 AV 포인트가 사용된다.

OUT Arguments - Complex Type

변수명	타입	비고
없음	없음	없음

7.13.4 examples

냉동기 1대에 냉각수를 공급하는 냉각탑 5대에 대해 26~28도일 경우 3대, 28~30도일 경우 4대, 30도 이상일 경우 5대를 동작시키고자 할 경우

- control : "제어 대수" (STEP CONTROL 함수에 의해 나온 결과값을 가지고 있는 제어 대수 관제점)
- st1 : "냉각탑상태1"
- ss1 : "냉각탑제어1"
- st2 : "냉각탑상태2"
- ss2 : "냉각탑제어2"
- st3 : "냉각탑상태3"

- ss3 : “냉각탑제어3”
- st4 : “냉각탑상태4”
- ss4 : “냉각탑제어4”
- st5 : “냉각탑상태5”
- ss5 : “냉각탑제어5”
- user1 : “냉각탑1번 사용유무”
- user2 : “냉각탑2번 사용유무”
- user3 : “냉각탑3번 사용유무”
- user4 : “냉각탑4번 사용유무”
- user5 : “냉각탑5번 사용유무”
- times : “시간 주기”

```
engine = ddc_logic()

engine.AUTO_ALTCONTROL5("제어 대수", "냉각탑상태 1", "냉각탑제어 1", "냉각탑상태 2",
"냉각탑제어 2", "냉각탑상태 3", "냉각탑제어 3", "냉각탑상태 4", "냉각탑제어 4", "냉각탑상태 5",
"냉각탑제어 5", "냉각탑 1 번 사용유무", "냉각탑 2 번 사용유무", "냉각탑 3 번 사용유무",
"냉각탑 4 번 사용유무", "냉각탑 5 번 사용유무", "시간주기")
```

7.13.5 연동 설비 개수별 지원 함수

- AUTO_ALTCONTROL5
- AUTO_ALTCONTROL10
- AUTO_ALTCONTROL14
- AUTO_ALTCONTROL16

7.14 Outdoor Reset Control

7.14.1 Usage

외기의 온도를 바탕으로 보정차를 두어 난방운전시 목표 온도를 변경함

7.14.2 Operation & In-out type

Operation Name	ORCONTROL(point, x0,y0,x1,y1)

외기온도에 따른 난방 목표 온도를 보정하기위해 사용한다. 외기 최저 온도에 따른 보정값과 외기 최고 온도에 따른 보정값을 설정하고 이 두값에 따른 1차방정식을 세운다. 외기온도가 최저,최고 온도 사이일 경우 이 1차방정식에 따라 보정값을 계산하여 난방 목표 온도를 설정할 수 있다. 최저,최고 온도의 범위를 넘어설 경우 최저,최고 온도에서 설정한 보정값이 변경없이 사용된다.

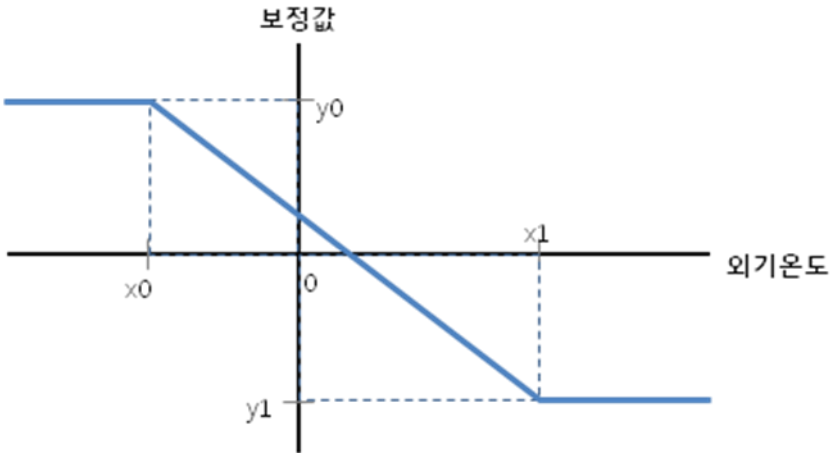


그림 7-8. 외기에 따른 보정 관계

IN Arguments - Complex Type

변수명	타입	비고
Outdoor temp	point	현재 외기 온도 관제점
x0	double	외기 최저 온도
y0	double	외기 최저 온도일 경우 보정값
x1	double	외기 최고 온도
y1	double	외기 최고 온도일 경우 보정값

OUT Arguments - Complex Type

변수명	타입	비고
output	double	보정치 연산 결과

7.14.3 examples

외기온도가 -10에서 15도까지 변동되고 -10도 일경우의 보정 편차 +5, 15도 일경우의 보정 편차 -5일 경우에 관리자의 목표 온도에 해당 보정치를 적용시키는 경우설정온도가 -5일 경우 제어 값이 0이고 설정온도가 -1일 경우 제어값이 4가 출력이 되야 하고 해당 값들의 관

계는 1차방정식의 관계를 가질 경우

- outdoor temp: “외기 온도”
- set temperature: “열교환기 설정온도”
- set temperature: “외기보상온도”

```
engine = ddc_logic()

orctemp = engine.ORCONTROL("외기 온도", -10, 5, 15, -5)
engine.WRITEPOINT("외기보상온도", orctemp + engine.READPOINT("열교환기 설정온도"))
```

7.15 Chilled Water Temperature Control

7.15.1 Usage

냉수출수설정온도 제어는 외기온도와 건물의 부하 상태에 따라 냉동기의 냉수출수온도의 설정점을 재조정 하는 제어를 말하며, 냉수출수온도를 고정적으로 설정하는 현장에 적용하면 소비전력을 줄일 수 있다.
일반적인 현장에서는 냉동기의 출수설정온도를 minimum으로 설정해놓고, 유지한다. 하지만 외기온도가 낮은 장마철이나 간절기에는 minimum 설정은 에너지 낭비가 될 수 있다. 외기온도와 건물부하(냉수입구온도 - 냉수출구온도)에 따라 설정온도를 조절함으로써 에너지 소모량을 줄일 수 있다.

7.15.2 Caution

로직 파일 당 1개의 함수만 사용해야 함.
조건문에 의해 수행되지 않는 경우 비정상 동작을 할 수 있음.

7.15.3 Operation & In-out type

Operation Name	LASER_CW_TEMP_APPLY(first,final,temp,humidity,x0,y0,x1,y1,cwout,cwin, cdwout,auto_p_value,auto_m_value,opermessage,trip_value,auto_gap, curr_status,final,period)

IN Arguments - Complex Type

변수명	타입	비고
first	point	로직이 적용된 1 차적인 설정온도. 일반적으로 AV 포인트가 사용된다.
final	point	보호로직까지 적용된 최종 냉수 출수 설정 온도. 일반적으로 AO 포인트가 사용된다.
temp	point	외기온도. 일반적으로 AI 포인트가 사용된다.
humidity	point	외기습도. 일반적으로 AI 포인트가 사용된다.
x0	point	외기온도 MIN(설정). 일반적으로 AV 포인트가 사용된다.
y0	point	냉동기 능력 MIN(설정) . 일반적으로 AV 포인트가 사용된다.
x1	point	외기온도 MAX(설정) . 일반적으로 AV 포인트가 사용된다.
y1	point	냉동기 능력 MAX(설정) . 일반적으로 AV 포인트가 사용된다.
cwout	point	현재 냉수출구온도. 일반적으로 AI 포인트가 사용된다.
cwin	point	현재 냉수입구온도. 일반적으로 AI 포인트가 사용된다.
cdwout	point	현재 냉각수출구온도. 일반적으로 AI 포인트가 사용된다.
auto_p_value	point	자동운전온도. 일반적으로 AV 포인트가 사용된다.
auto_m_value	point	자동정지온도. 일반적으로 AV 포인트가 사용된다.
opermessage	point	운전메시지. 일반적으로 AI 포인트가 사용된다.
trip_value	point	냉동기 트립 방지 값. 일반적으로 AV 포인트가 사용된다.
auto_gap	point	자동운전/정지 보정값. 일반적으로 AV 포인트가 사용된다.
curr_status	point	냉동기 상태(ON/OFF) . 일반적으로 BI 포인트가 사용된다.
final	point	현재 냉수출수설정온도. 일반적으로 AO 포인트가 사용되며.

		두번째 파라미터인 final 과 같은 포인트를 사용한다.
period	point	로직수행주기. 일반적으로 AV 포인트가 사용된다. 단위(분)

OUT Arguments - Complex Type

변수명	타입	비고
없음		

7.15.4 examples

Chapter 8.3 참조

7.16 Condenser Water Temperature Control

7.16.1 Usage

냉각수 설정 온도 제어는 냉각탑에서 냉동기로 들어오는 냉각수 입수의 온도를 제어 함으로써 냉각탑과 냉동기의 소비전력의 합이 최소가 되게 하는 최적제어를 함으로써 소비전력을 절감 할 수 있다.
일반 적인 현장에서는 냉각탑의 출수 설정온도를 고정으로 설정해놓고 유지한다. 냉동기 입장에서는 냉각탑이 낮은 온도의 냉각수를 보내주면 좋겠지만, 낮은 온도의 냉각수는 그만큼 냉각탑의 소비전력이 증가하게 된다.

7.16.2 Caution

로직 파일 당 1개의 함수만 사용해야 함.
조건문에 의해 수행되지 않는 경우 비정상 동작을 할 수 있음.

7.16.3 Operation & In-out type

Operation Name	$LASER_SEARCH_OPTIMAL_CT(double ct_status, double chiller_status, double ct_pump_status, double sT, double current_cdwin, double ct_power, double period)

IN Arguments - Complex Type

변수명	타입	비고
ct_status	point	냉각탑 상태. 일반적으로 DI 포인트가 사용된다.
chiller_status	point	냉동기 상태. 일반적으로 DI 포인트가 사용된다.
ct_pump_status	point	냉각탑 펌프 상태. 일반적으로 DI 포인트가 사용된다.
sT	point	냉수 출수 온도. 일반적으로 AI 포인트가 사용된다.
current_cdwin	point	현재 냉각수 입수온도. 일반적으로 AI 포인트가 사용된다.
ct_power	point	냉각탑 소비전력. 일반적으로 AV 포인트가 사용된다.
period	point	로직수행주기. 일반적으로 AV 포인트가 사용된다.

OUT Arguments - Complex Type

변수명	타입	비고
없음		

7.16.4 examples

Chaptor 8.4 참조

8 Sample Program

로직 작성 Guide

1. 요구사항분석
   A. 냉방시 XXX 제어, 난방시 YYY 제어, 연감지시 ZZZ 제어,,,,
   B. 일반 운전 사항 외적인 다른 시스템과의 연동 부분 고려
2. 요구사항에 대한 우선 순위 정렬
   A. 연감지 > 냉방, 난방 >
3. 요구사항 별 조건 도출
   A. 연기감지시, 냉방시, 난방시
4. 조건 내 행위 작성
   A. 댐퍼 CLOSE, 밸브 OFF,,,,
5. 각 조건 및 행위의 관제점 생성
   A. 냉난방운전, 공조기 운전 시,,,,
   B. 생성해야 하는 관제점을 타입별로 생성

로직 작성 주의점

1. 첫 라인에는 반드시 **“engine = ddc_logic()”** 구문이 존재해야 한다.
2. Indent(들여쓰기)를 반드시 지켜야 한다.
3. {} 시작하는 라인 끝에 **“:”**를 반드시 입력해야 한다.

8.1 AHU-NORMAL-TYPE

8.1.1 Definition

일반적인 H 형태의 공조기로서 급기,배기,환기,외기로 구성되는 공조기 중에서 급기/배기팬이 정풍량 type인 공조기를 AHU-NORMAL타입이라고 정의한다.

8.1.2 System Design

일반적으로 다음과 같은 형태로 구성된다.

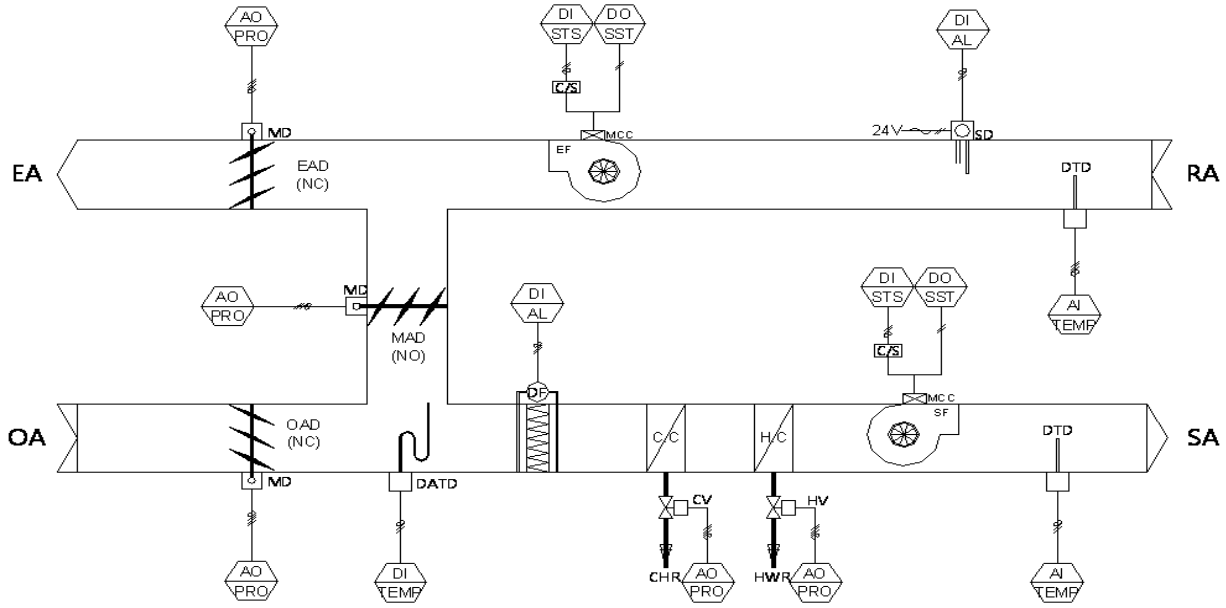


그림 8-1 정풍량 H 형 공조기

일반적으로 공조기 제어는 크게 3가지로 나뉘어 진다.

1. 댐퍼제어
2. 밸브제어
3. 팬제어

이런 세가지 제어가 복합적으로 이루어져서 여러가지 기능을 적용 시킬 수 있다.

8.1.3 Basic Control

일반적으로 다음과 같은 형태로 구성된다.

1. 밸브 제어
   A. 가장 기본이 되는 제어로서 온도에 따른 밸브제어를 한다.
   B. 공조기의 타입별로 상이한 부분이 있지만, 크게 난방시와 냉방시의 제어로 구분된다.
2. 팬 제어
   A. 공조기 운전의 근거로 사용되는 제어로서 모든 공조기의 로직은 급기팬 제어를 바탕으로 이루어 진다.
3. 댐퍼 제어
   A. 여러가지 지능적인 공조기 동작을 위한 제어 범위로서 일반적으로는 사용자가

설정값을 입력한다. 입력한 값은 외기 댐퍼의 제어 값으로 설정되며 배기는 같은 값, 혼합기는 역상으로 동작한다.

B. 이산화탄소 운전 및 엔탈피 제어 등의 지능적 공조기 동작은 댐퍼의 개도로 처리된다.

4. 기타 조건 제어
   A. 연감지 제어
      i. 연기감지 시의 제어로서 현장별로 요구사항이 상이함
   B. Warmming up 제어
      i. 난방시 빠른 온도 상승을 위한 방식, 밸브, 댐퍼제어
   C. 이산화탄소 제어
      i. 이산화탄소 값과 설정치를 비교하여 댐퍼제어
   D. 엔탈피 제어
      i. 외기 엔탈피와 내기 엔탈피를 비교하여 댐퍼제어

각 제어의 우선순위를 고려하여 조건문에 맞게 정렬하여 flow chart를 간단히 그리고 design 초안을 바탕으로 로직 작성을 한다.

8.1.4 Examples

정속 팬 타입의 일반적인 H형 공조기 (1-AHU -NORMAL)

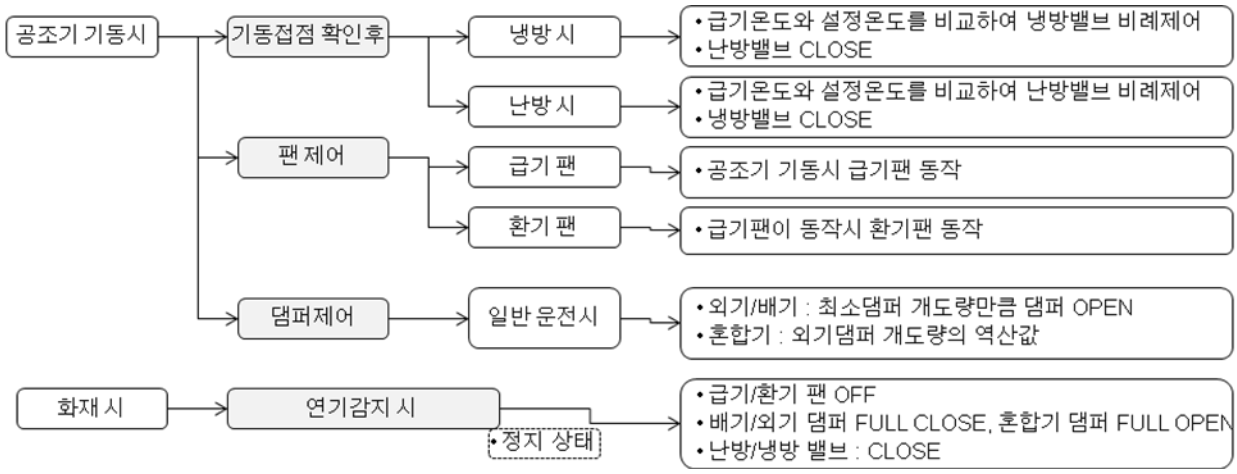


그림 8-2정풍량 H형 공조기의 요구사항 정리

8.2 AHU-INV-NORMAL

8.2.1 Definition

일반적인 H 형태의 공조기로서 급기,배기,환기,외기로 구성되는 공조기 중에서 급기/배기팬이 인버터 type인 공조기를 AHU-NORMAL타입이라고 정의한다.

8.2.2 System Design

일반적으로 다음과 같은 형태로 구성된다.

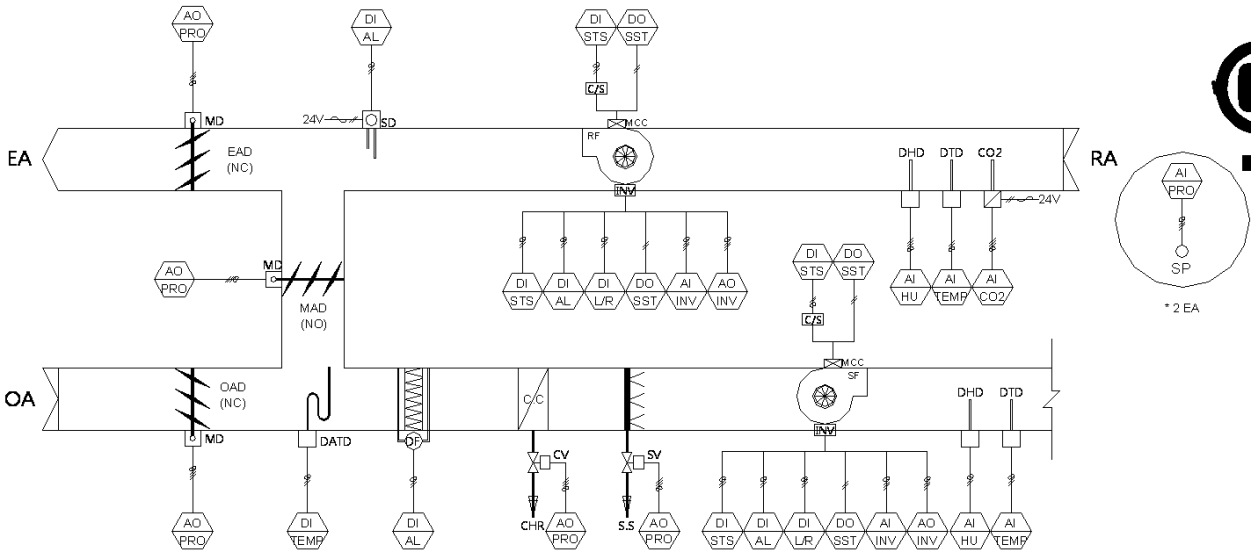


그림 8-3.일반적인 인버터 팬타입의 H형 공조기

일반적인 제어는 정풍량의 기본타입과 동일하다.

인버터 타입의 팬이 설치 되면 덕트내의 정압에 따른 제어가 추가된다.
급기팬과 환기팬의 유기적인 동작이 이루어지며 현장에 따라서 상이한 방식으로 운영된다.

8.2.3 Basic Control

일반적으로 다음과 같은 형태로 구성된다.

1. 밸브 제어
   A. 가장 기본이 되는 제어로서 온도에 따른 밸브제어를 한다.
   B. 공조기의 타입별로 상이한 부분이 있지만, 크게 난방시와 냉방시의 제어로 구분된다.
2. 팬 제어
   A. 공조기 운전의 근거로 사용되는 제어로서 모든 공조기의 로직은 급기팬 제어를 바탕으로 이루어 진다.
   B. 덕트내 정압센서에 의한 PID제어를 통해 균일 정압을 유지한다.
3. 댐퍼 제어
   A. 여러가지 지능적인 공조기 동작을 위한 제어 범위로서 일반적으로는 사용자가 설정값을 입력한다. 입력한 값은 외기 댐퍼의 제어 값으로 설정되며 배기는 같은 값, 혼합기는 역상으로 동작한다.

B. 이산화탄소 운전 및 엔탈피 제어 등의 지능적 공조기 동작은 댐퍼의 개도로 처리된다.

4. 기타 조건 제어
   A. 연감지 제어
      i. 연기감지 시의 제어로서 현장별로 요구사항이 상이함
   B. Warmming up 제어
      i. 난방시 빠른 온도 상승을 위한 방식, 밸브, 댐퍼제어
   C. 이산화탄소 제어
      i. 이산화탄소 값과 설정치를 비교하여 댐퍼제어
   D. 엔탈피 제어
      i. 외기 엔탈피와 내기 엔탈피를 비교하여 댐퍼제어

각 제어의 우선순위를 고려하여 조건문에 맞게 정렬하여 flow chart를 간단히 그리고 design 초안을 바탕으로 로직 작성을 한다.

8.2.4 Examples

인버터 팬 타입의 일반적인 H형 공조기 (1-AHU -INV-NORMAL)

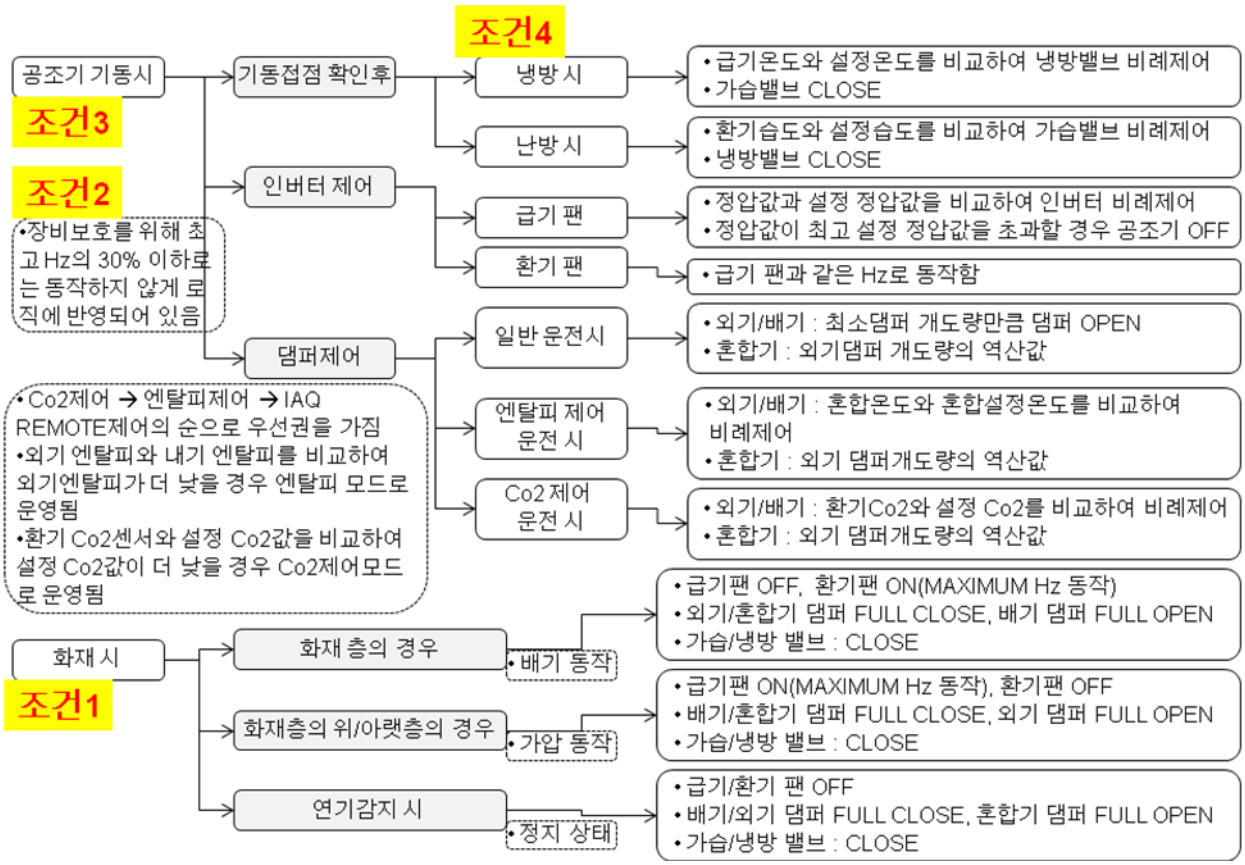


그림 8-4. 인버터타입의 H형 공조기 요구사항 정리

```
engine = ddc_logic()

#내부변수정의
nightpurge = 0
warnning = 0
```

```
#로직공통변수정의
ient = engine.ENTV("W35F.AHU.RATEMP", "W35F.AHU.RAHUM", 0)
oent = engine.READPOINT("W35F.AHU.OUTENT")
ohum = engine.READPOINT("W35F.AHU.OUTHUM")
engine.WRITEPOINT("W35F.AHU.INENT", ient)
engine.WRITEPOINT("W35F.AHU.SPAVG", engine.READPOINT("W35F.AHU.SP1"))

if(engine.READPOINT("W35F.AHU.NP.MODE") == 1):
    nightpurge = engine.NPURGE("W35F.AHU.NP.STIME", "W35F.AHU.NP.ETIME",
"W35F.AHU.NP.PERIOD", "W35F.AHU.RATEMP", "W35F.AHU.ROOMTEMP1",
"W35F.AHU.NP.DPTEMP", "W35F.AHU.NP.SETOTEMP", "W35F.AHU.NP.SETITEMP",
"W35F.AHU.NP.SETDPTEMP")

if(engine.READPOINT("W35F.FIRESTATUS") > 0):    #조건 1 : 화재시 운전
    engine.PRI(9, "W35F.AHU.SFINVOUT")
    engine.PRI(9, "W35F.AHU.SFSST")
    engine.PRI(9, "W35F.AHU.OADM")
    engine.PRI(9, "W35F.AHU.EFINVOUT")
    engine.PRI(9, "W35F.AHU.EFSST")
    engine.PRI(9, "W35F.AHU.EADM")
    engine.PRI(9, "W35F.AHU.MD")
    engine.PRI(9, "W35F.AHU.CV")
    engine.PRI(9, "W35F.AHU.SV")

    if(engine.READPOINT("W35F.FIRESTATUS") == 2):
        engine.WRITEPOINT("W35F.AHU.EFINVOUT", 10)
        engine.WRITEPOINT("W35F.AHU.EFSST", 1)
        engine.WRITEPOINT("W35F.AHU.EADM", 10)
        engine.WRITEPOINT("W35F.AHU.SFINVOUT", 0)
        engine.WRITEPOINT("W35F.AHU.SFSST", 0)
        engine.WRITEPOINT("W35F.AHU.OADM", 2)
        engine.WRITEPOINT("W35F.AHU.MD", 2)
        engine.WRITEPOINT("W35F.AHU.CV", 0)
        engine.WRITEPOINT("W35F.AHU.SV", 0)
    else:
        engine.WRITEPOINT("W35F.AHU.SFINVOUT", 10)
        engine.WRITEPOINT("W35F.AHU.SFSST", 1)
        engine.WRITEPOINT("W35F.AHU.OADM", 10)
        engine.WRITEPOINT("W35F.AHU.MD", 2)
        engine.WRITEPOINT("W35F.AHU.CV", 0)
        engine.WRITEPOINT("W35F.AHU.SV", 0)
```

```
        engine.WRITEPOINT("W35F.AHU.EFINVOUT", 0)
        engine.WRITEPOINT("W35F.AHU.EFSST", 0)
        engine.WRITEPOINT("W35F.AHU.EADM", 2)
else:
    if(engine.READPOINT("W35F.AHU.SPAVG") > engine.READPOINT("W35F.AHU.HIGHSP")):
        warnning = 1
    warnning = engine.READPOINT("W35F.AHU.SD") + warning

    if(warnning > 0): #조건 2 : 정압에 의한 운전
        engine.PRI(9, "W35F.AHU.SFINVOUT")
        engine.PRI(9, "W35F.AHU.SFSST")
        engine.PRI(9, "W35F.AHU.OADM")
        engine.PRI(9, "W35F.AHU.EFINVOUT")
        engine.PRI(9, "W35F.AHU.EFSST")
        engine.PRI(9, "W35F.AHU.EADM")
        engine.PRI(9, "W35F.AHU.MD")
        engine.PRI(9, "W35F.AHU.CV")
        engine.PRI(9, "W35F.AHU.SV")
        engine.WRITEPOINT("W35F.AHU.EFINVOUT", 0)
        engine.WRITEPOINT("W35F.AHU.EFSST", 0)
        engine.WRITEPOINT("W35F.AHU.EADM", 2)
        engine.WRITEPOINT("W35F.AHU.SFINVOUT", 0)
        engine.WRITEPOINT("W35F.AHU.SFSST", 0)
        engine.WRITEPOINT("W35F.AHU.OADM", 2)
        engine.WRITEPOINT("W35F.AHU.MD", 2)
        engine.WRITEPOINT("W35F.AHU.CV", 0)
        engine.WRITEPOINT("W35F.AHU.SV", 0)
        engine.WRITEPOINT("W35F.AHU.RUN", 0)
        engine.WRITEPOINT("W35F.AHU.SFSST", 0)
    else:
        engine.PRI(9, "W35F.AHU.RUN")

        if(nightpurge == 1):
            engine.WRITEPOINT("W35F.AHU.SFSST", 1)
        else:
            if(engine.READPOINT("W35F.AHU.RUN") == 1):
                engine.WRITEPOINT("W35F.AHU.SFSST", 1)
            else:
                engine.WRITEPOINT("W35F.AHU.SFSST", 0)

          if(engine.READPOINT("W35F.AHU.SFMGSTS") == 1): #조건 3-1: 공조기 운전시
```

```
        if(engine.READPOINT("W35F.AHU.HCMODE") == 0):     #조건 4-1: 냉방 운전시
            if(nightpurge == 1):
                engine.WRITEPOINT("W35F.AHU.CV", 0)
                engine.WRITEPOINT("W35F.AHU.OADM", 10)
                engine.WRITEPOINT("W35F.AHU.MD", 10)
                engine.WRITEPOINT("W35F.AHU.EADM", 10)
            else:
                engine.LOOP(1, "W35F.AHU.SATEMP", "W35F.AHU.CV", "W35F.AHU.SETTEMP",
1000, 20, 0, 0, 0, 0, 10, 0, 0)
                engine.WRITEPOINT("W35F.AHU.SV", 0)
        else: #조건 4-2: 난방 운전시
            if(engine.READPOINT("W35F.AHU.SAHUM") < 10):
                engine.LOOP(0, "W35F.AHU.RAHUM", "W35F.AHU.SV", "W35F.AHU.SETHUM", 1000,
20, 0, 0, 0, 0, 10, 1, 0)
            else:
                engine.WRITEPOINT("W35F.AHU.SV", 0)

            engine.WRITEPOINT("W35F.AHU.CV", 0)

        if(engine.READPOINT("W35F.AHU.CO2") > engine.READPOINT("W35F.AHU.SETCO2")):
#조건 4-2: Co2 제어시
            engine.LOOP(1, "W35F.AHU.CO2", "W35F.AHU.OADM", "W35F.AHU.SETCO2", 1000, 20,
0, 0, 2, 2, 10, 2, 0)
        else:
            if(engine.READPOINT("W35F.ENTHALPY") == 1): #조건 4-4: 엔탈피 제어시
                enthalpy = 0
                tempval = ient - 0.5
                if(oent < tempval):
                    enthalpy = 1

                tempval = ient + 0.5

                if(oent > tempval):
                    enthalpy = 0

                if(enthalpy == 1):
                    if(ohum <= 7):
                        engine.LOOP(1, "W35F.AHU.MTEMP", "W35F.AHU.OADM",
"W35F.AHU.SETMIXTEMP", 1000, 20, 0, 0, 2, 2, 10, 3, 0)
                    else:
                        engine.WRITEPOINT("W35F.AHU.OADM",
engine.READPOINT("W35F.AHU.MINSETDAMPER"))
```

```
                else:
                    engine.WRITEPOINT("W35F.AHU.OADM",
engine.READPOINT("W35F.AHU.MINSETDAMPER"))
            else:
                engine.WRITEPOINT("W35F.AHU.OADM",
engine.READPOINT("W35F.AHU.MINSETDAMPER"))

        tempval = engine.READPOINT("W35F.AHU.OADM")
        engine.WRITEPOINT("W35F.AHU.MD", tempval)
        engine.WRITEPOINT("W35F.AHU.EADM", tempval)
        bvalue = engine.READPOINT("W35F.AHU.MINOASETFAN")
        engine.LOOP(0, "W35F.AHU.SPAVG", "W35F.AHU.SFINVOUT", "W35F.AHU.SETSP", 500, 30,
0, 0, bvalue, bvalue, 10, 4, 0)
        engine.WRITEPOINT("W35F.AHU.EFSST", 1)
        engine.WRITEPOINT("W35F.AHU.EFINVOUT",
engine.READPOINT("W35F.AHU.SFINVOUT"))
    else: #조건 3-2: 공조기 정지시
        engine.LOOP_INIT(0)
        engine.LOOP_INIT(1)
        engine.LOOP_INIT(2)
        engine.LOOP_INIT(3)
        engine.LOOP_INIT(4)
        engine.WRITEPOINT("W35F.AHU.EFINVOUT", 0)
        engine.WRITEPOINT("W35F.AHU.EFSST", 0)
        engine.WRITEPOINT("W35F.AHU.SFINVOUT", 0)
        engine.WRITEPOINT("W35F.AHU.EADM", 2)
        engine.WRITEPOINT("W35F.AHU.OADM", 2)
        engine.WRITEPOINT("W35F.AHU.MD", 2)
        engine.WRITEPOINT("W35F.AHU.CV", 0)
        engine.WRITEPOINT("W35F.AHU.SV", 0)
```

8.3 Chilled Water Temperature Control

```
engine = ddc_logic()


if(engine.READPOINT("냉동기 1_에너지절감적용") == 1) #냉동기 에너지 절감 적용 시
    engine.LASER_CW_TEMP_APPLY("냉동기 1_출수온도가상값", "냉동기 1_냉수출구설정온도",
"외기온도", "외기습도", "냉동기 1_외기온도최소_하절기", "냉동기 1_냉수출수설정_MIN",
"냉동기 1_외기온도최대_하절기", "냉동기 1_냉수출수설정_MAX", "냉동기 1_냉수출구온도",
"냉동기 1_냉수입구온도", "냉동기 1_냉각수출구온도", "냉동기 1_자동운전온도",
"냉동기 1_자동정지온도", "냉동기 1_운전메시지",
"냉동기 1_트립방지설정값",     "냉동기 1_자동제어보정값", "냉동기 1_운전상태",
"냉동기 1_냉수출구설정온도", "냉동기 2_로직수행주기")
```

8.4 Condenser Water Temperature Control

```
engine = ddc_logic()

if(engine.READPOINT("냉각탑 1_에너지절감적용") == 1): #냉각탑 에너지 절감 적용 시
        engine.LASER_SEARCH_OPTIMAL_CT("냉각탑 1_운전상태", "냉동기 1_운전상태",
"냉동기 1_냉각수펌프상태",    "냉동기 1_냉수출구온도", "냉동기 1_냉각수입구온도",
"냉각탑 1_정격소비전력", "냉각탑 1_로직수행주기")
```